A Survey of Reinforcement Learning For Economics

Pranjal Rawat, Georgetown University

January 2026

Abstract

Dynamic programming provides the mathematical foundation for sequential decision-making under uncertainty, but the curse of dimensionality limits its practical reach. Reinforcement learning offers scalable alternatives, extending tractability to high-dimensional states, continuous actions, and strategic interactions. This survey introduces these methods to economists, reviewing the theory that connects dynamic programming to modern learning algorithms and demonstrating their power through worked examples in pricing, inventory control, strategic games, and preference elicitation. We also note the limits of reinforcement learning: lack of convergence guarantees, instability, brittleness, sensitivity to initialization and hyperparameters, and reliance on accurate simulators. A companion survey (Rust and Rawat, 2026b) covers the inverse problem: inferring preferences from observed behavior.

Keywords: Reinforcement Learning, Economics, Structural Estimation, Inverse Reinforcement Learning, Multi-Agent, Bandits, RLHF

1 Introduction

This survey introduces reinforcement learning to economists. We develop the theoretical connections between dynamic programming and reinforcement learning, demonstrating how value iteration, Q-learning, and policy gradient methods address variants of the same optimization problem. We then examine applications across several domains: control problems including pricing and inventory management; structural economic models with high-dimensional state spaces; strategic games in which multi-agent algorithms compute equilibria under imperfect information; bandit problems in which economic structure yields tighter regret bounds; and preference learning in which the Bradley-Terry model corresponds to discrete choice econometrics. The exposition combines formal development with computational illustration.

Dynamic programming provides the mathematical foundation for sequential decision-making under uncertainty. From optimal stopping to dynamic games, economists formulate choices as solutions to Bellman equations. The curse of dimensionality, however, constrains the scope of tractable problems. When state spaces are high-dimensional, action spaces continuous, or strategic interactions complex, backward induction becomes computationally infeasible.

Reinforcement learning extends dynamic programming to such settings. The underlying Bellman equations remain identical; the computational approach differs. Rather than computing value functions exactly over the entire state space, RL algorithms estimate them from sampled transitions. Rather than requiring explicit enumeration, RL approximates through function fitting. This methodological connection reflects parallel intellectual developments: dynamic programming in operations research evolved alongside reinforcement learning in artificial intelligence, with substantive exchange at several historical junctures.

These approximation methods sacrifice theoretical guarantees. RL algorithms lack the convergence assurances of exact dynamic programming. They exhibit sensitivity to hyperparameters and initialization. They can converge to suboptimal policies without diagnostic indication. This survey presents reinforcement learning as a computationally powerful framework while acknowledging its methodological limitations.

The survey addresses the forward problem: computing optimal policies given a known

or simulated environment. A companion survey treats the inverse problem of inferring preferences from observed behavior. Chapter 1 traces the parallel historical development of dynamic programming and reinforcement learning. Chapter 2 develops unified theory connecting planning and learning. Chapters 3 and 4 apply reinforcement learning to control problems and economic models. Chapter 5 examines strategic games. Chapter 6 addresses bandit problems. Chapter 7 discusses reinforcement learning from human feedback. Chapter 8 concludes.

2 A Brief History of Reinforcement Learning

This chapter offers a selective survey of key papers that inform the central debates and issues in reinforcement learning. Rather than exhaustive coverage, we focus on foundational contributions that shaped how the field thinks about learning, planning, and approximation. The narrative traces three intellectual threads that converged into modern RL. It then follows the classical synthesis from temporal difference learning through the identification of the deadly triad. Finally, it concludes with the deep learning era that revived and extended these ideas.

Notation

This chapter adopts a unified notation for concepts that developed independently across fields. States are denoted $s \in \mathcal{S}$, actions $a \in \mathcal{A}$, rewards r, and the discount factor $\gamma \in [0, 1)$. The state-value function is $V(s)$, the action-value function $Q(s, a)$, and a policy $\pi(a|s)$. The temporal difference error is $\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$. Learned weights are denoted $\mathbf{w}$ for linear models and θ for general parameterizations, with learning rate α. When presenting historical formalisms, we map them to this modern notation.

2.1 Roots

Modern reinforcement learning emerged from three distinct intellectual traditions: optimal control theory, game-playing programs, and animal learning psychology. Each contributed essential ideas that, taken together, define the field. We trace these threads from their origins through their eventual synthesis.

2.1.1 The Optimal Control Thread

Bellman (1957) Multi-stage decision problems require choosing today while accounting for how that choice constrains tomorrow. A warehouse manager deciding order quantities faces this structure: today's stock depends on yesterday's order, and tomorrow's depends on today's. Bellman (1957) recognized that all such problems share a recursive form.

A decision process consists of states $\mathcal{S}$, actions $\mathcal{A}$, transition kernel $P(s'|s,a)$, reward function $r(s,a)$, and discount factor γ. The optimal value function satisfies:

$$V^*(s) = \max_{a \in \mathcal{A}} \left[r(s,a) + \gamma \sum_{s'} P(s'|s,a)\, V^*(s') \right] \tag{1}$$

The principle of optimality states that an optimal policy has the property that, whatever the initial state and initial decision, the remaining decisions must constitute an optimal policy with respect to the state resulting from the first decision. Value iteration computes V^* by repeated application of the Bellman operator, converging at rate γ per iteration.[1] Bellman also identified the curse of dimensionality:[2] if the state $s = (x_1, \ldots, x_d)$ with each component x_i taking m values, then $|\mathcal{S}| = m^d$, and both storage and computation grow exponentially in the number of state variables.

Howard (1960) Value iteration updates every state on every iteration and converges geometrically at rate γ per iteration. Howard (1960) contributed policy iteration, which alternates between two steps and converges in finitely many iterations. The evaluation step computes V^π for the current policy π by solving the linear system

$$V^\pi(s) = r(s,\pi(s)) + \gamma \sum_{s'} P(s'|s,\pi(s))\, V^\pi(s') \quad \text{for all } s \in \mathcal{S} \tag{2}$$

and the improvement step updates the policy via

$$\pi'(s) = \underset{a \in \mathcal{A}}{argmax} \left[r(s,a) + \gamma \sum_{s'} P(s'|s,a)\, V^\pi(s') \right] \tag{3}$$

[1] The Bellman operator T maps value functions to value functions via $(TV)(s) = \max_a [r(s,a) + \gamma \sum_{s'} P(s'|s,a) V(s')]$. Convergence at rate γ means $\|V_k - V^*\|_\infty \leq \gamma^k \|V_0 - V^*\|_\infty$.

[2] The curse of dimensionality refers to exponential growth of state space with dimension. For d continuous variables discretized into m bins each, $|\mathcal{S}| = m^d$, making exact methods infeasible in high dimensions.

Howard proved that policy iteration converges to the optimal policy in a finite number of steps.

The Bellman equation and Howard's policy iteration together constitute the planning framework for sequential decisions under uncertainty. This framework assumes complete knowledge of P and r. The challenge of learning optimal behavior without such knowledge is the central problem of reinforcement learning. Chapter 3 develops the relationship between planning and learning in detail.

2.1.2 The Game Engines Thread

Shannon (1950) Chess provides a well-defined domain for studying machine intelligence: precise rules, a clear objective, and a need for planning, evaluation, and strategic reasoning. Shannon (1950) posed the fundamental question: can we program a general-purpose computer to play chess, and if so, what principles should guide the design?

The challenge is computational. A chess game averages 40 moves per player with roughly 30 legal moves available at each position. Exhaustive search through all possible games would require examining approximately $30^{80} \approx 10^{120}$ positions. Brute-force enumeration fails at the scale of the game tree. Human masters examine only a few dozen positions when choosing a move, guided by pattern recognition. But encoding this expertise into explicit rules proved difficult.

Shannon distinguished two approaches. Type A strategies search all continuations to a fixed depth; Type B strategies search selectively, examining only important variations. Either approach requires an evaluation function $f(P)$ and a move selection rule. The minimax principle governs adversarial search: assume the opponent plays optimally. Shannon proposed evaluating positions as weighted sums of features:

$$f(P) = \sum_i w_i \phi_i(P) \tag{4}$$

Features included material, mobility, pawn structure, and king safety.[3] Shannon did not implement a chess program; the paper is theoretical, outlining the architecture and key design choices.

This framework shaped all subsequent game-playing AI. Deep Blue (1997) used pri-

[3]The minimax principle states that the value of a position in a zero-sum game is $V(s) = \max_a \min_{a'} U(s, a, a')$ where the maximizing player moves first and the minimizing opponent responds optimally.

marily Type A search with hardware parallelism; modern engines like Stockfish combine deep Type A search with alpha-beta pruning; AlphaZero (2017) revived Type B ideas through neural network move prioritization. The question of how to learn or optimize evaluation weights, which Shannon did not address, became the central problem for Samuel, Tesauro, and the deep reinforcement learning era.

Samuel (1959) Samuel (1959) built a checkers program for the IBM 704 that could improve through experience rather than manual tuning. The program learned through self-play using two complementary mechanisms.

Rote learning stored every board position encountered along with its backed-up value, creating an expanding lookup table. Generalization learning evaluated positions with a linear function

$$V(s) = \sum_i w_i \phi_i(s) \tag{5}$$

where $\phi_i(s)$ were hand-crafted features: piece advantage, piece advancement, center control, mobility, and king-row threat. After each move, the weights were updated according to

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha \left[V(s_{t+1}) - V(s_t) \right] \nabla_{\mathbf{w}} V(s_t) \tag{6}$$

This update adjusts the current position's value toward the value of the successor position. The program reached expert-level play by 1962 and defeated Robert Nealey, a self-proclaimed expert, in a publicized match. Samuel's update is TD(0) with linear function approximation, a connection formalized by Sutton (1988) twenty-nine years later.[4]

Checkers was well-suited to Samuel's approach because domain experts had already identified meaningful board features, reducing the function approximation problem to learning a linear combination of informative inputs. The moderate branching factor ($b \approx 8$) kept the search tree manageable, and self-play generated unlimited training data at no cost.

[4] Alpha-beta pruning, implemented following McCarthy's suggestion, reduces the minimax search complexity from $O(b^d)$ to $O(b^{d/2})$ under optimal move ordering. For checkers ($b \approx 8$, $d = 10$), this reduces the search space from $8^{10} \approx 10^9$ to $8^5 \approx 33{,}000$ positions.

2.1.3 The Animal Psychology Thread

Thorndike (1898) Comparative psychology in the 1890s sought to measure animal intelligence through controlled experiment rather than anecdote. Thorndike (1898) placed hungry cats inside latched wooden boxes with food visible outside and recorded escape latency across repeated trials. Trial 1 required 160 seconds of undirected clawing, biting, and squeezing through slats. By trial 24 the cat pressed the latch directly in 7 seconds. The learning curve was smooth rather than stepwise, ruling out sudden insight and suggesting incremental strengthening of stimulus-response associations.[5] Thorndike codified this pattern as the Law of Effect: of several responses made to the same situation, those which are accompanied or closely followed by satisfaction to the animal will, other things being equal, be more firmly connected with the situation, so that, when it recurs, they will be more likely to recur. In modern terms, the box configuration is the state s, the motor action is a, and escape followed by food is the reward r. The stamping in of successful responses increases the probability of selecting that action given the state, a directional update rule that Q-learning would formalize nine decades later.

Rescorla-Wagner (1972) Classical conditioning theory through the mid-twentieth century held that temporal pairing of stimuli was sufficient for associative learning. The blocking experiment overturned this view by demonstrating that prediction error, not co-occurrence, drives associative change. The protocol proceeds in two phases. In Phase I, a rat learns that a tone predicts shock and develops a conditioned fear response to the tone alone. In Phase II, the compound stimulus of tone plus light is paired with the same shock. When the light is subsequently presented alone, no fear response occurs. The light is blocked because the tone already predicts the shock perfectly, leaving no prediction error to drive learning about the light.

Rescorla and Wagner (1972) formalized this observation:

$$\Delta V_i = \alpha_i \beta_j (\lambda_j - V_{\text{tot}}) \tag{7}$$

where V_i is the associative strength of stimulus i, α_i is the salience of stimulus i, β_j is the learning rate associated with unconditioned stimulus j, λ_j is the maximum conditioning

[5] Thorndike's escape-time data followed an approximate power law $T(n) \propto n^{-\beta}$ rather than an exponential decay, a pattern later termed the power law of practice.

that stimulus j supports, and $V_{\text{tot}} = \sum_k V_k$ is the total prediction from all stimuli present on the trial.[6]

Learning halts when $V_{\text{tot}} = \lambda_j$. In blocking, the tone already predicts the shock perfectly ($V_A = \lambda$), so the prediction error is zero and the added light acquires no associative strength.

The Rescorla-Wagner model maps directly to the temporal difference framework. With a discount factor $\gamma = 0$ in a single-step episode, the prediction error $(\lambda_j - V_{\text{tot}})$ is equivalent to the TD error $\delta = r - V(s)$. The model is a special case of TD learning for single-step, terminal-reward tasks.

2.2 The Classical Synthesis

2.2.1 Sutton (1988)

The gap between dynamic programming, which requires a complete model of the environment, and Monte Carlo estimation, which requires waiting for complete episodes, motivated a method that could learn incrementally from partial experience. Sutton (1988) formalized temporal difference learning and demonstrated its properties on a five-state random walk. States A through E are arranged in a line with absorbing terminal states at each end. At each step the agent moves left or right with equal probability. The right terminal yields reward 1 and the left terminal yields reward 0. The true state values are $\frac{1}{6}, \frac{2}{6}, \frac{3}{6}, \frac{4}{6}, \frac{5}{6}$. Sutton compared TD(0), which updates after every transition, with Monte Carlo estimation, which updates only at the end of each episode. TD(0) converged faster and with less data.

The general TD(λ) update combines both approaches through an eligibility trace:

$$V(s_t) \leftarrow V(s_t) + \alpha\, \delta_t\, e_t(s) \tag{8}$$

where $\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$ is the TD error and $e_t(s) = \gamma\lambda\, e_{t-1}(s) + \mathbb{1}\{s = s_t\}$ is the eligibility trace for state s. Setting $\lambda = 0$ yields pure bootstrapping; setting $\lambda = 1$ recovers Monte Carlo returns. Intermediate values interpolate, balancing the low variance of TD(0) against the low bias of Monte Carlo.

[6] The Rescorla-Wagner update is mathematically identical to the Widrow-Hoff (1960) least mean squares rule used in adaptive signal processing. Both minimize mean squared prediction error by stochastic gradient descent.

2.2.2 Watkins (1989)

Temporal difference methods as developed by Sutton solve the prediction problem: estimating the value of a given policy. The control problem requires finding the optimal policy without access to the transition dynamics $P(s'|s,a)$. Dynamic programming computes optimal policies via the Bellman optimality equation, but this requires a complete model of the MDP. The question is whether the same optimality can be achieved from sampled transitions alone.

TD(0) learns value functions $V(s)$, but converting these to actions still requires knowing transition probabilities: given $V(s')$ for all successor states, the agent needs to know which action leads to which successor. Model-based approaches learn $P(s'|s,a)$ from experience and then apply dynamic programming, but this introduces approximation errors at both stages.

Watkins and Dayan (1992), formalizing Watkins's 1989 PhD thesis, provided the solution: learn $Q(s,a)$ directly, the expected return from taking action a in state s and then behaving optimally. The optimal policy is then $\pi^*(s) = \mathit{argmax}_a\, Q^*(s,a)$, requiring no model to act. The Bellman optimality equation provides the fixed-point condition:

$$Q^*(s,a) = \mathbb{E}\left[r + \gamma \max_{a'} Q^*(s',a') \mid s,a\right] \tag{9}$$

Q-learning achieves this via incremental updates:

$$Q(s_t,a_t) \leftarrow Q(s_t,a_t) + \alpha\left[r_{t+1} + \gamma \max_{a'} Q(s_{t+1},a') - Q(s_t,a_t)\right] \tag{10}$$

The Bellman operator $\mathcal{T}$ is a contraction with modulus γ, ensuring a unique fixed point $Q^* = \mathcal{T}Q^*$. Convergence requires finite state and action spaces, bounded rewards, and Robbins-Monro learning rates. The maximization over a' makes Q-learning off-policy:[7] the update target uses the greedy action at the next state regardless of the action actually taken. The agent can follow an ε-greedy exploration strategy while learning about the optimal policy directly. Chapter 3 develops the contraction property and convergence theory in detail.

[7] Off-policy learning means the agent learns about a policy different from the one generating data. Q-learning learns about the greedy policy $\pi^*(s) = \mathit{argmax}_a\, Q(s,a)$ while collecting data with an exploratory policy.

2.2.3 Williams (1992)

Value-based methods learn an action-value function and derive a policy from it. Williams (1992) derived an alternative: policy gradients that optimize the policy directly by gradient ascent on expected return:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t)\, G_t\right] \tag{11}$$

where $G_t = \sum_{k=0}^{T-t} \gamma^k r_{t+k+1}$ is the discounted return from time t. The log-derivative trick allows the gradient to be estimated from sampled trajectories without differentiating through the environment dynamics.

Consider a robot arm that must apply a continuous torque $a \in \mathbb{R}$ to reach a target angle. Q-learning requires computing $\max_a Q(s, a)$ at every update, which becomes a nested optimization problem when the action space is continuous. A policy gradient method sidesteps the issue: parameterize the policy as a Gaussian $\pi_\theta(a|s) = \mathcal{N}(\mu_\theta(s),\, \sigma_\theta^2(s))$,[8] sample an action, observe the return, and update θ by REINFORCE. No maximization over actions is required.[9] The practical difficulty is variance. Because G_t is a sum of stochastic rewards, gradient estimates can be noisy. Variance reduction techniques, including baselines $b(s_t)$ subtracted from G_t, are essential.[10]

2.2.4 Tesauro (1994)

Tesauro (1994)'s TD-Gammon demonstrated that temporal difference learning with neural network function approximation could produce expert-level play. Backgammon has approximately 10^{20} legal positions, far beyond tabular methods.

TD-Gammon trained a feedforward neural network to estimate the probability of winning from any board position. A hidden layer of 80 sigmoid units fed a single sigmoid output:

$$\hat{V}(s) = \sigma\Big(\mathbf{w}^\top \sigma(W\mathbf{x}(s) + \mathbf{b}) + c\Big) \tag{12}$$

where σ is the logistic sigmoid, W is the input-to-hidden weight matrix, and $\mathbf{w}$ is the

[8] The Gaussian distribution $\mathcal{N}(\mu, \sigma^2)$ has density $(2\pi\sigma^2)^{-1/2} \exp(-(a-\mu)^2/2\sigma^2)$. Here $\mu_\theta(s)$ and $\sigma_\theta(s)$ are neural network outputs parameterizing the policy mean and standard deviation.

[9] This is why virtually all continuous-control results in reinforcement learning descend from the policy gradient framework rather than from Q-learning.

[10] The variance-minimizing baseline is approximately $V^\pi(s_t)$, leading to the advantage actor-critic architecture where $G_t - b(s_t) \approx A^\pi(s_t, a_t)$.

hidden-to-output weight vector.

The network was trained by self-play using TD(λ) with $\lambda = 0.7$. After each move from position s_t to s_{t+1}, the weights $\boldsymbol{\theta}$[11] were updated by

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha\Big[\hat{V}(s_{t+1}) - \hat{V}(s_t)\Big]\,\mathbf{e}_t \tag{13}$$

where the eligibility trace $\mathbf{e}_t = \sum_{k=1}^{t} \lambda^{t-k} \nabla_{\boldsymbol{\theta}} \hat{V}(s_k)$ accumulates exponentially decayed gradients of past predictions.[12] At game's end, $\hat{V}(s_{t+1})$ is replaced by the outcome $z \in \{0, 1\}$.

TD-Gammon 1.0, trained on 300,000 self-play games, played at a strong intermediate level. Version 2.1 achieved near-parity with Bill Robertie, a former world champion. The program discovered novel positional strategies subsequently adopted by the human backgammon community. Backgammon was uniquely favorable: stochastic dice ensured exploration, the value function varied smoothly with board position, and self-play generated unlimited on-policy training data, avoiding the off-policy component of the deadly triad.

2.2.5 Baird (1995)

Baird (1995) constructed a seven-state star MDP demonstrating divergence of Q-learning with linear function approximation. The MDP has six outer states that all transition to a single inner state under the target policy. The off-policy behavior samples states uniformly. With linear function approximation, the weights grow without bound under repeated Q-learning updates.

The source of instability is the interaction of three components: bootstrapping (updating from estimated values rather than observed returns), off-policy learning (training on data from a different policy than the target), and function approximation (representing the value function with a parameterized model). Sutton and Barto (2018) later named this the deadly triad. Any two components can be combined safely; all three together permit divergence.[13]

[11] We use $\boldsymbol{\theta}$ to denote the full vector of network weights and biases, generalizing the scalar θ used for policy parameters in earlier sections.

[12] In the neural network case, the eligibility trace $\mathbf{e}_t \in \mathbb{R}^{|\boldsymbol{\theta}|}$ is a vector accumulating exponentially-weighted gradients, extending the scalar state-based trace to parameter space.

[13] The divergence arises because the TD fixed point minimizes projected Bellman error $\|V_\theta - \Pi T V_\theta\|$. When the projection and Bellman operator interact adversarially under off-policy sampling, the fixed point may not

This explains the asymmetry between Tesauro's success and Baird's failure. TD-Gammon used bootstrapping and function approximation but was on-policy: training data came from the same self-play policy whose value was being estimated. Baird's counterexample used all three components and diverged. Baird also proposed a constructive solution: residual gradient algorithms that perform gradient descent on the mean-squared Bellman residual, guaranteeing convergence at the cost of a different fixed point.

2.3 The Modern Era

2.3.1 Deep Q-Networks (2015)

The two decades after TD-Gammon saw limited success in applying neural networks to reinforcement learning. Attempts to replicate Tesauro's approach in chess and Go did not achieve comparable results, though progress continued in robotics and other domains. Mnih et al. (2015) changed this by training a single convolutional neural network to play 49 Atari 2600 games directly from pixel inputs ($210 \times 160 \times 3$) and a scalar score, using no game-specific features.

The architecture processed four consecutive frames through three convolutional layers and a fully connected layer. The network $Q(s, a; \theta)$ was trained to minimize the squared temporal difference error:

$$L(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta)\right)^2\right] \tag{14}$$

Two innovations stabilized learning. Experience replay stored transitions (s, a, r, s') in a buffer $\mathcal{D}$ holding 10^6 transitions and sampled uniformly for training. This breaks the temporal correlation between consecutive updates that would otherwise cause the network to overfit to recent experience. A target network used a frozen copy of parameters θ^-,[14] updated to match θ only every $C = 10{,}000$ steps, so the regression target does not shift with each gradient step.

These techniques address the deadly triad identified by Baird. Experience replay creates a more stationary data distribution, partially mitigating off-policy issues. The target network holds the bootstrap target fixed between updates, making the regression

exist or may be unstable.

[14] The notation θ^- denotes target network parameters, a lagged copy of θ updated periodically. This stabilizes learning by keeping the regression target fixed during gradient steps.

problem approximately stationary. Neither technique provides theoretical guarantees, but the empirical success demonstrated that the deadly triad can be managed in practice.

DQN exceeded human-level performance on 29 of 49 games using a single architecture and hyperparameters. The same network that learned Breakout (where the agent must track a ball and position a paddle) also learned Space Invaders (where the agent must track multiple enemies and time shots). Games requiring long-horizon planning or sparse rewards, such as Montezuma's Revenge (where the agent must navigate rooms and collect keys before receiving any reward), remained difficult.

2.3.2 TRPO and PPO (2015, 2017)

Policy gradient methods suffer from a practical instability: a single large gradient step can move the policy into a region of parameter space where performance collapses, and recovery may be slow or impossible. The objective $J(\theta)$ is not well approximated by its linear Taylor expansion when the step size is large relative to the curvature of the policy space.

Schulman et al. (2015) addressed this with Trust Region Policy Optimization (TRPO). TRPO solves the constrained optimization problem

$$\max_{\theta} \; L_{\theta_{\text{old}}}(\theta) \quad \text{subject to} \quad \bar{D}_{\text{KL}}(\theta_{\text{old}}, \theta) \leq \delta \tag{15}$$

where L is a surrogate objective based on the advantage function $A^{\pi}(s,a) = Q^{\pi}(s,a) - V^{\pi}(s)$.[15] The KL constraint prevents the policy from moving so far in one update that performance collapses and recovery becomes impossible.

Schulman et al. (2017) proposed Proximal Policy Optimization (PPO) as a simpler alternative. PPO replaces the KL constraint with a clipped surrogate objective:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min\Big(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon)\hat{A}_t \Big) \right] \tag{16}$$

where $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$ is the probability ratio. The clipping removes the incentive for $r_t(\theta)$ to move outside the interval $[1-\varepsilon, 1+\varepsilon]$, penalizing large policy updates without explicitly computing a divergence measure.

[15] The Kullback-Leibler divergence $D_{\text{KL}}(p\|q) = \sum_x p(x)\log(p(x)/q(x))$ measures statistical distance between distributions. The bar denotes expectation over states: $\bar{D}_{\text{KL}} = \mathbb{E}_s[D_{\text{KL}}(\pi_{\theta_{\text{old}}}(\cdot|s)\|\pi_\theta(\cdot|s))]$.

PPO outperformed A2C, TRPO, and the cross-entropy method on continuous control benchmarks and achieved the highest average reward on 30 of 49 Atari games among the methods tested. PPO became the default policy optimization algorithm for large-scale RL applications. The lesson from both TRPO and PPO: constraining the magnitude of policy updates is essential for stable optimization.

2.3.3 AlphaGo and AlphaGo Zero (2016, 2017)

The game of Go, with approximately 10^{170} legal positions and a branching factor of roughly 250, had resisted both brute-force search and hand-crafted evaluation functions. Silver et al. (2016) combined learning and planning by integrating multiple components: a policy network trained by supervised learning on 30 million positions from human expert games, a second policy network improved by self-play reinforcement learning, a value network trained to predict game outcomes from positions, and Monte Carlo tree search (MCTS) guided by these networks. AlphaGo defeated Lee Sedol, one of the strongest Go players in history, four games to one in March 2016. Go was well-suited to this architecture because its fixed 19×19 board is naturally suited to convolutional networks, its perfect information and deterministic transitions make MCTS's tree structure exact, and the game outcome provides a clean binary training signal. Silver et al. (2017) removed human data entirely. AlphaGo Zero uses a single neural network $f_\theta(s) = (\mathbf{p}, v)$ that takes a board position s as input and outputs a policy vector $\mathbf{p}$ over legal moves and a scalar value v estimating the probability of winning. The training loop is self-contained: the program plays games against itself using MCTS guided by f_θ, collects the MCTS-improved move probabilities $\boldsymbol{\pi}$ and the game outcome $z \in \{-1, +1\}$, then updates f_θ to minimize

$$\ell(\theta) = (z - v)^2 - \boldsymbol{\pi}^\top \log \mathbf{p} + c\|\theta\|^2 \tag{17}$$

where the first term is a value prediction loss, the second is a policy cross-entropy loss, and the third is L_2 regularization. After 72 hours of self-play on 4 TPUs, AlphaGo Zero surpassed all previous versions of AlphaGo, including the version that defeated Lee Sedol. Igami (2020) interprets the AlphaGo architecture in econometric terms: the policy network is a conditional choice probability (CCP) estimator, the value network is a conditional value function (CVF) estimator, and the system performs CCP estimation

and forward simulation jointly, connecting to the Hotz-Miller tradition in economics.

2.3.4 Toward a Unified Framework

The common object underlying all these developments is the Bellman operator. Value iteration, policy iteration, Q-learning, and policy gradients all seek fixed points of operators derived from the Bellman equation. The difference between planning and learning is whether that operator is applied with known transitions or sampled ones. Model-based methods learn the transition dynamics explicitly; model-free methods estimate value functions directly from experience. Both approaches converge to the same optimal policy under appropriate conditions, but they differ in sample complexity, computational requirements, and sensitivity to model misspecification. Chapter 3 develops this unified framework in detail.

3 Theory of Planning and Learning

4 Planning and Learning

4.1 Introduction

Economists solve dynamic programs. The Bellman equation appears in consumption-savings problems, job search, investment timing, and structural estimation of discrete choice models. Standard treatments in **?** and Rust (1987) assume the transition probabilities and reward functions are known; the computational task is to find the fixed point of the Bellman operator. Reinforcement learning solves the same fixed-point problem but under two additional constraints: the transition model may be unknown (requiring learning from sampled transitions), and the state space may be too large for exact computation (requiring function approximation). This chapter develops the theoretical foundations that govern when and why RL algorithms converge.

The central insight is that RL algorithms are not mysterious: they are asymptotic approximations to classical dynamic programming operators. Value iteration becomes Q-learning when expectations are replaced by single samples. Policy iteration becomes the natural policy gradient when the greedy improvement step is approximated by gradient ascent. The stochastic approximation framework of **?** provides the mathematical

foundation: under appropriate step-size conditions, noisy iterates converge to the same fixed points as their deterministic counterparts.

Table 1 summarizes the six sections of this chapter, each motivated by a question that arises when moving from textbook dynamic programming to data-driven learning.

Table 1: Chapter roadmap: from dynamic programming to reinforcement learning

§	Title	Economist's Question	Key Resolution
2.2	The Newton Connection	Why does PI converge faster than VI?	PI = Newton's method
2.3	Stochastic Approximation	Can we replace $\mathbb{E}$ with one sample?	Robbins-Monro; Q-learning
2.4	The Deadly Triad	Why does regression inside Bellman diverge?	Projection geometry
2.5	Policy Gradient	Can we optimize π directly without V?	Policy gradient theorem
2.6	Actor-Critic	How to reduce variance of policy gradient?	Two-timescale learning
2.7	Bridging Results	What are the error bounds?	Singh-Yee; sparse sampling

4.2 The Newton Connection

Economists learn value iteration (VI) and policy iteration (PI) from **?**: VI applies the Bellman operator repeatedly until convergence; PI alternates between policy evaluation (solving a linear system) and policy improvement (taking the greedy action). The standard observation is that PI converges faster. But why? The answer reveals a deep connection between dynamic programming and numerical optimization: policy iteration is Newton's method applied to the Bellman equation.

Consider the Bellman optimality operator T acting on value functions:

$$(TV)(s) = \max_{a \in \mathcal{A}} \left\{ r(s,a) + \gamma \sum_{s' \in \mathcal{S}} P(s'|s,a) V(s') \right\}. \tag{18}$$

This operator is nonlinear due to the max. Value iteration applies T repeatedly: $V_{k+1} = TV_k$. Since T is a γ-contraction in the supremum norm, Banach's fixed-point theorem guarantees $\|V_k - V^*\|_\infty \leq \gamma^k \|V_0 - V^*\|_\infty$. This is Picard iteration, with linear convergence at rate γ.

Policy iteration takes a different approach. At the current value estimate $\tilde{V}$, define the greedy policy $\tilde{\pi}(s) = argmax_a \{r(s,a) + \gamma \sum_{s'} P(s'|s,a)\tilde{V}(s')\}$. The policy evaluation step solves the linear fixed-point equation $V = T^{\tilde{\pi}} V$ exactly, where $T^{\tilde{\pi}}$ is the policy-specific Bellman operator:

$$(T^{\tilde{\pi}} V)(s) = r(s, \tilde{\pi}(s)) + \gamma \sum_{s'} P(s'|s, \tilde{\pi}(s)) V(s'). \tag{19}$$

The key insight, formalized by **?**, is geometric: the linear operator $T^{\tilde{\pi}}$ is a supporting hyperplane to the nonlinear operator T at the current iterate. Specifically:

1. Tangency: $T^{\tilde{\pi}}\tilde{V} = T\tilde{V}$ (the operators agree at the current point).
2. Support: $T^{\tilde{\pi}}V \leq TV$ for all V (the linear operator lies below the nonlinear one everywhere).

Policy evaluation solves for the fixed point of this linearization exactly. This is precisely the structure of Newton's method: linearize the nonlinear equation at the current point, solve the linearized system, and iterate.[16]

The consequence is quadratic convergence near the optimum. While VI requires $k = \log(100)/\log(1/\gamma)$ iterations to reduce error by a factor of 100, PI typically converges in 5–10 iterations regardless of γ. Table 2 illustrates the difference.

Table 2: Iterations for 100× error reduction: VI vs. PI

γ	Value Iteration	Policy Iteration
0.90	44	5–8
0.95	90	5–8
0.99	459	5–10

? extends this interpretation to a broad class of dynamic programming problems. The Newton structure applies whenever the Bellman operator can be written as a pointwise maximum over linear operators: $T = \max_\pi T^\pi$. This includes not only discounted MDPs but also stochastic shortest path problems, average-cost MDPs, and model predictive control formulations. The practical implication is that algorithms with policy-improvement structure (evaluate a policy exactly or approximately, then improve) inherit Newton-like convergence behavior, while pure value-iteration methods (apply T directly) are limited to linear convergence.

4.3 Stochastic Approximation

The Bellman operator in Equation (18) requires computing an expectation: $\mathbb{E}_{s'\sim P(\cdot|s,a)}[V(s')]$. In structural estimation, this typically means either knowing the transition matrix P explicitly or integrating over a high-dimensional distribution. But what if we only observe sampled transitions (s, a, r, s') from an unknown environment? Can a single noisy sample replace the expectation and still yield convergence?

[16]The Newton interpretation of policy iteration has precursors in **?** for Riccati equations in linear-quadratic control and **?** for stochastic games.

The answer is yes, and the mathematical foundation is stochastic approximation, developed by **?**. Consider the problem of finding x^* such that $g(x^*) = 0$, where we cannot evaluate g directly but can observe noisy samples $g(x) + \epsilon$. The Robbins-Monro iteration is:

$$x_{t+1} = x_t - \alpha_t[g(x_t) + \epsilon_t], \tag{20}$$

where ϵ_t is zero-mean noise. Under two conditions on the step sizes, this converges to x^* with probability one:

1. $\sum_{t=0}^{\infty} \alpha_t = \infty$ (sufficient exploration: we never stop learning),
2. $\sum_{t=0}^{\infty} \alpha_t^2 < \infty$ (diminishing noise: the variance of cumulative updates is finite).

The canonical choice $\alpha_t = 1/(t+1)$ satisfies both conditions.

Q-learning (Watkins and Dayan, 1992) is Robbins-Monro applied to the Bellman equation for action-value functions. Define the Q-factor Bellman operator:

$$(FQ)(s,a) = r(s,a) + \gamma \sum_{s'} P(s'|s,a) \max_{a'} Q(s',a'). \tag{21}$$

The Q-learning update, upon observing transition (s_t, a_t, r_t, s_{t+1}), is:

$$Q(s_t,a_t) \leftarrow Q(s_t,a_t) + \alpha_t \left[r_t + \gamma \max_{a'} Q(s_{t+1},a') - Q(s_t,a_t) \right]. \tag{22}$$

The term in brackets is a noisy sample of $(FQ)(s_t,a_t) - Q(s_t,a_t)$, the Bellman residual. The noise arises because we observe a single next state s_{t+1} rather than the full distribution $P(\cdot|s_t,a_t)$.

Theorem 1 (Q-Learning Convergence)**.** *Under the conditions: (i) every state-action pair (s,a) is visited infinitely often, (ii) step sizes satisfy the Robbins-Monro conditions $\sum_t \alpha_t(s,a) = \infty$ and $\sum_t \alpha_t(s,a)^2 < \infty$ for each (s,a), Q-learning converges to Q^* with probability one.*

This result, established by Watkins and Dayan (1992) and **?**, shows that model-free learning is possible: we can find the optimal value function without ever estimating the transition probabilities.

SARSA provides an on-policy variant. Instead of taking the maximum over next

actions, SARSA uses the action actually taken:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha_t \left[r_t + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t) \right]. \tag{23}$$

This solves for the value function of the behavior policy π rather than the optimal policy. Singh et al. (2000) prove convergence under the same step-size conditions, provided the behavior policy converges to a stationary distribution.

Q-learning is noisy value iteration on Q-factors; SARSA is noisy policy evaluation. The Robbins-Monro conditions ensure that the noise averages out faster than the signal decays, allowing asymptotic convergence despite using only single-sample estimates.

Rollout methods extend this framework by performing one step of policy improvement from a base policy. Given a base policy μ with value function V^μ, the rollout policy selects:

$$\tilde{\pi}(s) = \underset{a}{argmax} \left\{ r(s,a) + \gamma \sum_{s'} P(s'|s,a) V^\mu(s') \right\}. \tag{24}$$

? proves the cost improvement property: $V^{\tilde{\pi}}(s) \leq V^\mu(s)$ for all s, with strict inequality unless μ is already optimal. AlphaZero implements this principle at scale: neural networks provide V^μ, and Monte Carlo Tree Search extends the effective lookahead. Each MCTS iteration is a sampled rollout, and the aggregation across many samples approximates the expectation in the Bellman operator.[17]

4.4 The Deadly Triad

The convergence results of the previous section assume tabular representations: a separate value for each state-action pair. When the state space is large or continuous, this is infeasible. The standard remedy is function approximation: represent $V(s) \approx \hat{V}(s;\theta)$ using a parameterized family such as polynomials, splines, or neural networks. Regression is then used to fit θ from sampled Bellman targets. This combination of regression and dynamic programming, however, can diverge.

Temporal difference (TD) learning with function approximation does not minimize Bellman error directly. Instead, it finds a fixed point of the projected Bellman operator ΠT^π, where Π projects onto the function approximation subspace. Consider linear func-

[17] The error bound for H-step lookahead with terminal approximation V^μ is $\|V^{\tilde{\pi}} - V^*\|_\infty \leq \frac{2\gamma^H}{1-\gamma} \|V^\mu - V^*\|_\infty$. At $H = 50$ and $\gamma = 0.99$, this multiplier is approximately 1.2; longer lookahead compensates for worse value approximations.

tion approximation $\hat{V}(s;\theta) = \phi(s)^\top \theta$, where $\phi(s) \in \mathbb{R}^d$ is a feature vector. The projection Π maps any value function to its best approximation in the span of the features, under some norm.

Tsitsiklis and Van Roy (1997) establish the convergence conditions. Define the d^π-weighted norm $\|V\|_{d^\pi}^2 = \sum_s d^\pi(s) V(s)^2$, where d^π is the stationary distribution of the Markov chain induced by policy π. When samples are drawn from d^π (on-policy sampling), the projection Π is non-expansive in this norm: $\|\Pi V_1 - \Pi V_2\|_{d^\pi} \leq \|V_1 - V_2\|_{d^\pi}$. Since T^π is a γ-contraction in any weighted ℓ_∞ norm, the composition ΠT^π remains a contraction, and TD converges to the unique fixed point.

Off-policy sampling breaks this argument. Baird (1995) constructs a counterexample: a six-state MDP with two-dimensional linear function approximation, where samples are drawn uniformly rather than from d^π. The TD updates cause the weight vector to diverge to infinity. The geometric intuition is that uniform sampling induces a different norm under which Π is no longer non-expansive; the composition ΠT^π ceases to be a contraction and can amplify errors.

This phenomenon is called the deadly triad: the combination of function approximation, bootstrapping (TD targets that depend on current estimates), and off-policy learning. Removing any one element restores convergence:

1. Without function approximation (tabular), the projection Π is the identity and convergence follows from Section 4.3.
2. Without bootstrapping (Monte Carlo returns), targets are independent of current estimates and regression is well-posed.
3. Without off-policy learning (on-policy sampling), the Tsitsiklis-Van Roy conditions are satisfied.

DQN (Mnih et al., 2015) addresses the deadly triad through two mechanisms. Experience replay stores transitions in a buffer and samples mini-batches uniformly, breaking temporal correlation and making the sampling distribution approximately stationary. Target networks hold the regression target fixed for many updates: instead of regressing toward $r + \gamma \max_{a'} Q(s', a'; \theta)$, DQN regresses toward $r + \gamma \max_{a'} Q(s', a'; \theta^-)$, where θ^- is updated slowly. This decouples the moving target problem from the parameter updates.[18]

[18] **?** reformulate TD as a saddle-point problem, yielding gradient TD methods that converge off-policy with linear function approximation. These methods are less commonly used in practice than target networks.

4.5 Policy Gradient

Value-based methods find fixed points of the Bellman operator. Policy-based methods take a different approach: parameterize the policy directly as $\pi_\theta(a|s)$ and maximize expected return $J(\theta) = \mathbb{E}_{\pi_\theta}[\sum_{t=0}^{\infty} \gamma^t R_t]$ by gradient ascent. This formulation sidesteps the Bellman equation entirely and frames reinforcement learning as constrained optimization.

The policy gradient theorem (**?**) provides a tractable expression for the gradient:

$$\nabla_\theta J(\theta) = \mathbb{E}_{s\sim d^{\pi_\theta}, a\sim\pi_\theta} \left[\nabla_\theta \log \pi_\theta(a|s)\, Q^{\pi_\theta}(s,a)\right], \tag{25}$$

where $d^{\pi_\theta}(s) = (1-\gamma)\sum_{t=0}^{\infty} \gamma^t \mathbb{P}(s_t = s|\pi_\theta)$ is the discounted state visitation distribution. The gradient does not require $\nabla_\theta d^{\pi_\theta}$, the derivative of the state distribution with respect to policy parameters. This term is intractable in general, but it cancels in the derivation.

REINFORCE (Williams, 1992) is the simplest policy gradient algorithm. Sample a trajectory $(s_0, a_0, r_0, s_1, \ldots)$, compute the return $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$ from each time step, and update:

$$\theta \leftarrow \theta + \alpha \sum_t \nabla_\theta \log \pi_\theta(a_t|s_t)\, G_t. \tag{26}$$

This is an unbiased estimator of $\nabla_\theta J(\theta)$, but its variance is high because a single trajectory provides a noisy estimate of Q^{π_θ}.

Standard gradient descent treats all parameter directions equally. But small changes in θ can cause large changes in the policy distribution π_θ. The natural policy gradient (**?**) accounts for this curvature by preconditioning with the Fisher information matrix:

$$\tilde{\nabla}_\theta J(\theta) = F(\theta)^{-1}\nabla_\theta J(\theta), \quad F(\theta) = \mathbb{E}_{s,a}\left[\nabla_\theta \log \pi_\theta(a|s) \nabla_\theta \log \pi_\theta(a|s)^\top\right]. \tag{27}$$

In the tabular setting, NPG is equivalent to policy iteration: the natural gradient direction exactly matches the policy improvement step. This explains why NPG inherits the fast convergence of PI.

The RL objective $J(\theta)$ is non-convex, raising concerns about local optima. **?** establish that for softmax policies, $J(\theta)$ satisfies a gradient domination condition:

$$\|\nabla_\theta J(\theta)\|_2^2 \geq \frac{(1-\gamma)^3}{8}\left(J(\pi^*) - J(\pi_\theta)\right). \tag{28}$$

This Polyak-Łojasiewicz-type inequality implies that every stationary point is globally optimal. NPG with constant step size $\eta = (1-\gamma)/4$ achieves $J(\pi^*) - J(\pi_k) \leq O(1/k)$, with no dependence on $|\mathcal{S}|$ or $|\mathcal{A}|$.

The performance difference lemma provides the foundation for trust region methods. For any two policies π and π':

$$J(\pi') - J(\pi) = \frac{1}{1-\gamma}\mathbb{E}_{s\sim d^{\pi'}}\left[\sum_a \pi'(a|s)A^\pi(s,a)\right], \tag{29}$$

where $A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$ is the advantage function. This identity underlies TRPO (Schulman et al., 2015) and PPO (Schulman et al., 2017), which constrain policy updates to regions where the advantage estimates remain accurate.

Adding entropy regularization accelerates convergence. The entropy-regularized objective is:

$$J_\tau(\theta) = \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{\infty}\gamma^t\left(R_t + \tau\mathcal{H}(\pi_\theta(\cdot|s_t))\right)\right], \tag{30}$$

where $\mathcal{H}(\pi) = -\sum_a \pi(a)\log\pi(a)$ is the entropy and $\tau > 0$ is the temperature. **?** prove that NPG with entropy regularization achieves linear convergence: $J_\tau(\pi^*) - J_\tau(\pi_k) \leq (1-c\tau)^k \cdot [J_\tau(\pi^*) - J_\tau(\pi_0)]$ for some constant $c > 0$. The optimal policy under entropy regularization is softmax: $\pi^*(a|s) \propto \exp(Q^*(s,a)/\tau)$, which matches the logit choice probability from discrete choice econometrics.

4.6 Actor-Critic

REINFORCE uses the sample return G_t to estimate the policy gradient. This estimator is unbiased but has high variance: a single trajectory provides a noisy signal. TD methods use bootstrapped estimates $r + \gamma V(s')$ as targets. These have lower variance (the value function aggregates information across trajectories) but are biased when V is approximate. Actor-critic methods combine both: the critic estimates the value function, the actor updates the policy using the critic's estimates.

The theoretical foundation is two-timescale stochastic approximation (**?**). Run two

concurrent learning processes:

$$\text{Critic (fast):} \quad \theta_{t+1} = \theta_t + \alpha_t^{(c)} \delta_t \nabla_\theta \hat{V}(s_t; \theta_t), \tag{31}$$

$$\text{Actor (slow):} \quad \omega_{t+1} = \omega_t + \alpha_t^{(a)} \delta_t \nabla_\omega \log \pi_\omega(a_t | s_t), \tag{32}$$

where $\delta_t = r_t + \gamma \hat{V}(s_{t+1}; \theta_t) - \hat{V}(s_t; \theta_t)$ is the TD error. The critic updates the value function parameters θ; the actor updates the policy parameters ω.

Convergence requires the critic to learn faster than the actor:

$$\lim_{t \to \infty} \frac{\alpha_t^{(a)}}{\alpha_t^{(c)}} = 0, \quad \text{with both satisfying Robbins-Monro conditions.} \tag{33}$$

Under this separation, the actor sees a quasi-stationary critic: from the actor's perspective, the critic provides approximately correct value estimates at each step. The actor's updates are then approximately unbiased policy gradient steps. **?** prove convergence to a local optimum of $J(\omega)$.

A2C (Advantage Actor-Critic) is the synchronous variant: collect a batch of transitions, compute TD errors, and update both networks. A3C (**?**) parallelizes this across multiple workers updating a shared parameter server asynchronously. SAC (Soft Actor-Critic) (**?**) adds entropy regularization, using the soft Bellman operator for the critic and entropy-augmented returns for the actor.

The actor-critic structure separates identification from optimization. The critic solves a regression problem (estimate V^π from data), while the actor solves an optimization problem (improve π using the estimated values). This decomposition mirrors the structure of many econometric estimators: first estimate the model, then compute optimal decisions given the estimates.

4.7 Bridging Results

The preceding sections establish that RL algorithms converge under appropriate conditions. Two questions remain: how does approximation error in the value function propagate to the policy, and how does computational complexity scale with the problem size?

? bound the policy degradation from value function errors. If $\hat{V}$ approximates V^* with error $\|\hat{V} - V^*\|_\infty \leq \epsilon$, and $\hat{\pi}$ is the greedy policy with respect to $\hat{V}$, then:

$$\|V^* - V^{\hat{\pi}}\|_\infty \leq \frac{2\gamma}{1-\gamma}\epsilon. \tag{34}$$

At $\gamma = 0.99$, the amplification factor is $2 \cdot 0.99/0.01 = 198$. A 1% error in value function approximation yields at most 198% error in policy value. This bound is pessimistic but finite: approximate value functions do not cause unbounded policy degradation.

Classical dynamic programming complexity scales with the state space size $|\mathcal{S}|$. For problems like Go, where $|\mathcal{S}| \approx 10^{170}$, exact computation is impossible. **?** prove that with access to a generative model (a simulator that samples transitions from any state-action pair), near-optimal planning requires samples polynomial in:

1. The effective horizon $H = O(\log(1/\epsilon)/\log(1/\gamma))$,
2. The branching factor $|\mathcal{A}|$,
3. The desired accuracy $1/\epsilon$.

The sample complexity has no dependence on $|\mathcal{S}|$. The sparse sampling algorithm builds a random tree of depth H from the current state, estimates values by averaging leaf rewards, and selects the action with highest estimated value. This result explains why Monte Carlo Tree Search succeeds in large state spaces: planning complexity depends on the horizon and action space, not the state space cardinality.

The apparent dichotomy between model-based and model-free RL dissolves under closer examination. Model-based methods learn a transition model $\hat{P}$ and use it to generate synthetic experience; model-free methods use real experience directly. Dyna (**?**) unifies both: after each real transition (s, a, r, s'), perform k additional updates using transitions simulated from a learned model. In the limit of infinite simulated transitions, the model-based and model-free approaches converge to the same solution. The practical tradeoff is sample efficiency (model-based methods extract more information per real sample) versus model bias (errors in $\hat{P}$ can compound).

Table 3 summarizes the quantitative results from this chapter.

Table 3: Summary of convergence results

Result	Rate/Bound	Implication
VI convergence	γ^k	460 iterations for 100× reduction at $\gamma = 0.99$
PI convergence	Quadratic	5–10 iterations for same
Q-learning	$\tilde{O}(1/\sqrt{t})$	$\sim 10^6$ samples for $\epsilon = 0.01$
Value error → policy	$\frac{2\gamma}{1-\gamma}\epsilon$	Bounded degradation
Sparse sampling	No $\lvert\mathcal{S}\rvert$ dependence	Curse of dimensionality broken for planning
NPG	$O(1/k)$	No $\lvert\mathcal{S}\rvert$, $\lvert\mathcal{A}\rvert$ dependence

4.8 Conclusion

Reinforcement learning is not a departure from dynamic programming but an extension of it. The algorithms in this chapter are asymptotic approximations to the Bellman fixed-point operator, justified by the mathematics of contractions, stochastic approximation, and gradient domination. Policy iteration is Newton's method; Q-learning is noisy value iteration; natural policy gradient recovers policy iteration in the tabular case. The theoretical framework provides guarantees: convergence rates, error bounds, and conditions under which approximation is safe.

Several open problems remain. The sample complexity of Q-learning exceeds the minimax lower bound by a factor of $(1-\gamma)^{-1}$, and closing this gap is an active area of research. Function approximation theory for deep networks remains incomplete: the deadly triad is well-characterized for linear approximation, but less is known about the implicit regularization provided by neural network architectures. Model-based methods show promise for sample efficiency, but theoretical guarantees for learned world models are limited. These gaps define the frontier between the algorithms we understand and the algorithms we use.

5 Applications of Reinforcement Learning

This chapter constructs nine economic benchmarks that span major application domains where reinforcement learning has been deployed in production or studied extensively. For each benchmark, we review the deployment literature, define a formal MDP, present the Bellman equation, and compare RL against practitioner heuristics. The benchmarks are organized by domain: pedagogical foundations, durable goods replacement, transportation, pricing and marketing, revenue management, auctions, finance, supply chain, and energy.

5.1 Example 1: Gridworld Navigation (Pedagogical)

We begin with two pedagogical examples that validate the benchmark methodology before applying it to economically meaningful domains. The gridworld is the canonical example in reinforcement learning, appearing in virtually every textbook since Sutton and Barto (2018). While lacking direct economic application, it provides a clean testbed for algorithm validation: the MDP is small enough for exact dynamic programming yet rich enough to require multi-step planning.

We use a deterministic $N \times N$ gridworld where an agent navigates from the origin to a terminal goal state. The agent receives a small negative reward at each step, creating pressure to reach the goal quickly. This simple structure admits closed-form solutions while still requiring multi-step planning.

Definition D1 (Gridworld Navigation MDP)**.** *The gridworld MDP $\mathcal{M}_{\text{grid}}(N)$ is defined as follows.*

1. *State space. $\mathcal{S} = \{0, \ldots, N-1\}^2$, representing grid positions. A state $s = (r, c)$ is a row-column coordinate. The state space size is $|\mathcal{S}| = N^2$.*
2. *Action space. $\mathcal{A} = \{\textit{Left}, \textit{Right}, \textit{Up}, \textit{Down}, \textit{Stay}\}$, with $|\mathcal{A}| = 5$.*
3. *Terminal state. $s^* = (N-1, N-1)$ is the absorbing goal state.*
4. *Transition function. Deterministic: action a moves the agent in the corresponding direction, clipped to stay within grid boundaries. Formally, for action a with direction vector $\delta_a \in \{-1, 0, 1\}^2$:*

$$T(s, a) = \text{clip}(s + \delta_a, [0, N-1]^2). \tag{35}$$

5. *Reward function.*

$$r(s, a) = \begin{cases} R_{\text{goal}} & \textit{if } T(s, a) = s^*, \\ -c_{\text{step}} & \textit{otherwise}, \end{cases} \tag{36}$$

   where $R_{\text{goal}} = 10$ is the terminal reward and $c_{\text{step}} = 0.1$ is the step penalty.
6. *Discount factor. $\gamma = 0.95$.*

The Bellman optimality equation is

$$V^*(s) = \max_{a \in \mathcal{A}} \left[r(s, a) + \gamma \, V^*(T(s, a)) \right]. \tag{37}$$

The optimal policy follows the shortest path to the goal. With $N = 10$, the state space has 100 states, and Value Iteration converges in under 0.1 seconds.

The heuristic baseline is greedy Manhattan navigation: at each step, move in the direction that minimizes Manhattan distance to the goal.

Placeholder: results pending simulation run

Figure 1: Gridworld benchmark ($N = 10$). Left: wall-clock time comparison. Right: learning curve showing DQN convergence to the DP optimal value.

Table 4: Gridworld benchmark results ($N = 10$, 100 states).

Method	Reward	Time (s)	Convergence	Transitions	Entropy	DP Agr.
DP (VI)	8.30	0.16	316 iter	—	0.00	100%
DQN	8.30 ± 0.00	48.1	1005 ep	84k	1.60	80.0%
Greedy	8.30	—	—	—	—	—

The gridworld validates the benchmark methodology: DQN converges to the DP optimal policy, confirming correct implementation before applying the same evaluation to economically meaningful domains.

5.2 Example 2: Bus Engine Replacement (Economics)

The bus engine replacement problem is the canonical example in economics, introduced by Rust (1987) to estimate dynamic discrete choice models. A fleet manager decides each period whether to replace a bus engine based on its accumulated mileage. Replacement incurs a fixed cost but resets mileage to zero; continued operation incurs maintenance costs that increase with mileage.

We extend the single-engine problem to a fleet of N engines with a capacity constraint limiting how many can be replaced per period. This extension preserves the essential economic tradeoff while creating state space complexity that scales exponentially in N.

Definition D2 (Multi-Engine Bus Replacement MDP)**.** *The multi-engine bus replacement MDP $\mathcal{M}_{\text{bus}}(N)$ is defined as follows.*

1. *State space.* $\mathcal{S} = \{0, \ldots, M-1\}^N$*, where M is the number of mileage bins and N is the number of engines. A state $s = (m_1, \ldots, m_N)$ records discretized mileage for each engine. The state space size is $|\mathcal{S}| = M^N$.*
2. *Action space.* $\mathcal{A}$ *is the set of subsets of $\{1, \ldots, N\}$ with cardinality at most C (the capacity constraint). With $C = 3$, the action space includes replacing any combination of 0, 1, 2, or 3 engines.*
3. *Transition function. Deterministic mileage evolution: replaced engines reset to $m = 0$; non-replaced engines increment by one bin (capped at $M-1$).*
4. *Cost function. Per-period cost is*

$$c(s, a) = \alpha \cdot \sum_{i=1}^{N} \mathbb{1}\{m_i > 0\} + \beta \cdot |a|, \tag{38}$$

   where α is the per-engine operating cost for engines with positive mileage, β is the replacement cost per engine, and $|a|$ is the number of engines replaced.
5. *Reward function.* $r(s, a) = -c(s, a)$.
6. *Discount factor.* $\gamma = 0.95$.

The Bellman optimality equation is

$$V^*(s) = \max_{a \in \mathcal{A}} \left[-c(s, a) + \gamma V^*(s') \right], \tag{39}$$

where s' is the deterministic next state. With $M = 6$ mileage bins, the state space grows as 6^N: 1,296 states at $N = 4$, rising to 7,776 at $N = 5$ and 46,656 at $N = 6$. Value Iteration is feasible for $N \leq 5$.

The heuristic set contains two policies. The threshold policy replaces all engines with mileage ≥ 3 (up to capacity). The never-replace policy never replaces any engine.

The bus engine benchmark connects RL to economics. The optimal policy balances immediate replacement costs against the long-run maintenance burden of high-mileage engines, a tradeoff that threshold heuristics approximate but do not optimize.

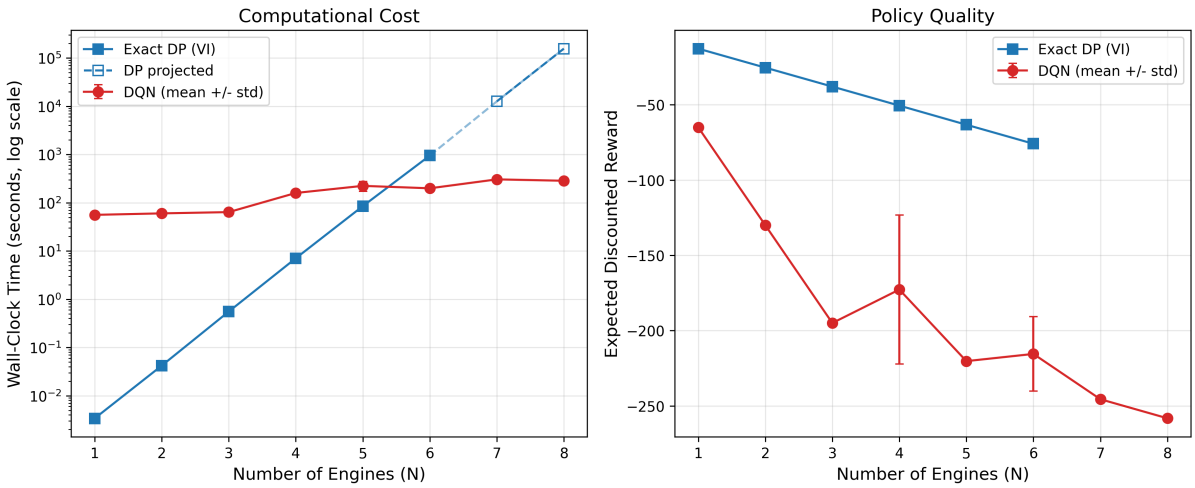

Figure 2: Bus engine replacement benchmark ($N = 4$). Left: wall-clock time comparison. Right: learning curve showing DQN convergence to DP optimal.

Table 5: Bus engine replacement benchmark results across fleet sizes N.

N	$\lvert\mathcal{S}\rvert$	DP Time (s)	DP Reward	DQN Time (s)	DQN Reward	DQN Bellman Res.
1	6	0.00	-12.72	56.01 ± 1.59	-65.01 ± 0.03	0.0027 ± 0.0003
2	36	0.04	-25.33	60.14 ± 1.25	-130.00 ± 0.02	0.0141 ± 0.0057
3	216	0.55	-37.92	63.87 ± 9.83	-194.95 ± 0.05	0.0664 ± 0.0252
4	1,296	7.05	-50.55	159.21 ± 3.56	-172.71 ± 49.39	0.0569 ± 0.0468
5	7,776	84.85	-63.25	223.62 ± 50.47	-220.25 ± 0.04	0.1147 ± 0.0887
6	46,656	947.50	-75.88	199.08 ± 24.71	-215.47 ± 24.71	0.1318 ± 0.0062
7	279,936	—	—	304.73 ± 2.61	-245.57 ± 0.05	0.1797 ± 0.1049
8	1,679,616	—	—	285.11 ± 7.00	-258.22 ± 0.05	1.5799 ± 1.5623

5.3 Benchmark 3: Transportation and Logistics

Ride-hailing platforms face a real-time matching problem: assign drivers to passengers to maximize platform efficiency. The problem is sequential because each assignment changes the spatial distribution of available drivers, affecting future match quality.

Qin et al. (2021) formalized DiDi's ride-hailing dispatch as a large-scale multi-agent RL problem. The state space encodes driver locations, passenger request queues, and time-of-day features across a city partitioned into hexagonal zones. Actions are driver-to-request assignments. The reward combines driver income, passenger wait time, and order completion rates. Their deployed system achieved 0.5–2% improvements in key metrics including driver income and order completion rates compared to previous production baselines, and the peer-reviewed evaluation confirmed that the RL dispatcher consistently outperformed combinatorial optimization baselines across multiple Chinese cities.[19] At DiDi's scale of tens of millions of daily rides, these percentage gains translate to substantial aggregate efficiency improvements.

Scaling RL dispatch to city-wide operations introduces a coordination challenge: thousands of drivers must be assigned simultaneously, and each driver's optimal action depends on what other drivers do. Li et al. (2019) addressed this by applying mean-field multi-agent RL, where each driver conditions on the aggregate distribution of other drivers rather than their individual states. This approximation makes the multi-agent problem tractable while preserving the essential spatial interaction structure: a driver's expected pickup time depends on how many other drivers are nearby, which the mean-field distribution captures. Experiments on real DiDi data showed improved fleet utilization compared to single-agent RL baselines.

Han et al. (2022) reported on RL deployment at Lyft, providing a complementary perspective from a different platform and market. Their system optimizes driver dispatching and positioning jointly, using a value decomposition architecture that assigns credit to individual driver-level decisions within the global objective. The production deployment demonstrated improvements in both rider wait times and driver utilization, confirming that the RL dispatch paradigm generalizes across platforms and regulatory environments.

We formalize the dispatch problem as a zone-based MDP that captures the essential

[19] An earlier technical report (Wang et al., 2019b) described the system at a high level; Qin et al. (2021) provides the formal treatment and controlled experiments.

spatial allocation structure of these deployments.

Definition D3 (Zone-Based Dispatch MDP)**.** *The zone-based dispatch MDP* $\mathcal{M}_{\text{disp}}(K)$ *is defined as follows.*

1. *State space.* $\mathcal{S} = \{0, \ldots, D_{\max}\}^K \times \{0, \ldots, Q_{\max}\}^K$*, where* $D_{\max}$ *is the maximum idle drivers per zone,* $Q_{\max}$ *is the maximum demand queue per zone, and* K *is the number of zones. A state* $s = (d_1, \ldots, d_K, q_1, \ldots, q_K)$ *records idle drivers and queued demand per zone. The state space size is* $|\mathcal{S}| = (D_{\max} + 1)^K \cdot (Q_{\max} + 1)^K$*.*
2. *Action space.* $\mathcal{A} = \{0, 1, \ldots, K\}$*, with* $|\mathcal{A}| = K + 1$*. Action* 0 *is no rebalancing; action* $k \in \{1, \ldots, K\}$ *rebalances one driver from zone* k *to the zone with the highest demand queue.*
3. *Transition function. Each period: (i) new demand arrives at zone* k *as a Poisson random variable with rate* λ_k*, added to the queue; (ii) matches occur as* $\min(d_k, q_k)$ *per zone; (iii) matched drivers return to the idle pool next period; (iv) any rebalancing action transfers one driver between zones.*
4. *Reward function.*

$$r(s, a) = f \cdot \sum_{k=1}^{K} \min(d_k, q_k) - c_r \cdot \mathbb{1}\{a > 0\}, \tag{40}$$

   where f *is the fare per match and* c_r *is the rebalancing cost.*
5. *Discount factor.* $\gamma = 0.95$*.*

The Bellman optimality equation is

$$V^*(s) = \max_{a \in \mathcal{A}} \left[\mathbb{E}[r(s, a)] + \gamma \sum_{s'} P(s'|s, a)\, V^*(s') \right]. \tag{41}$$

The complexity grows as $(D_{\max} + 1)^K \cdot (Q_{\max} + 1)^K$, which is exponential in K. With $D_{\max} = Q_{\max} = 5$, the state space reaches 6^{2K}: 1,296 states at $K = 2$, rising to over 60 million at $K = 5$. Value Iteration is feasible for $K \leq 3$ and infeasible beyond.

The heuristic set $\mathcal{H}$ contains three policies. The no-rebalance policy never moves drivers between zones, relying entirely on organic arrivals to clear demand queues. The nearest-match heuristic rebalances one driver from the zone with the most idle drivers to the zone with the highest demand queue, a greedy spatial matching rule. The proportional heuristic rebalances toward zones where the demand-to-driver ratio is highest,

approximating a fluid model solution.

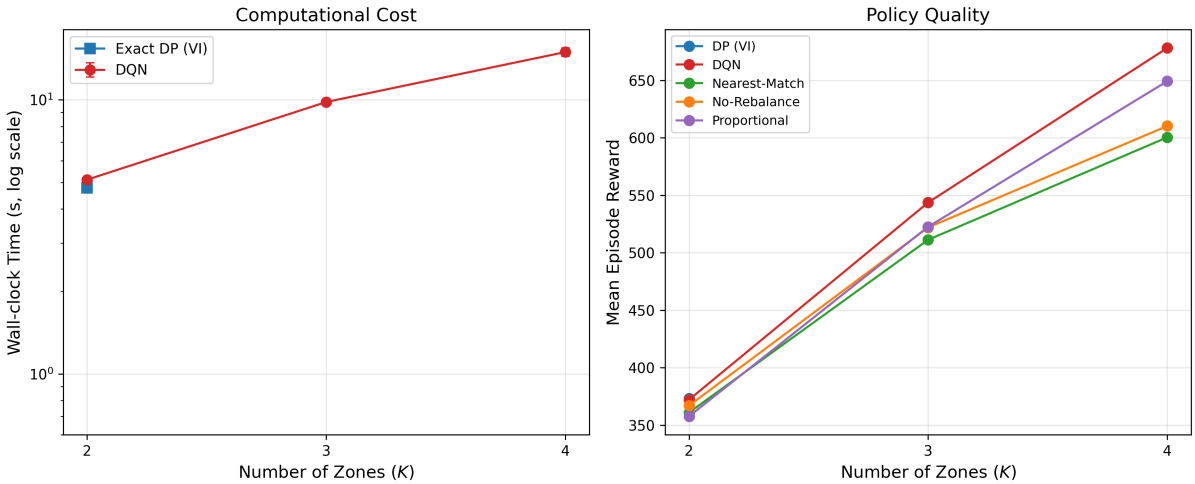

Figure 3: Dispatch benchmark. Left: wall-clock time (log scale) vs. number of zones K. Right: mean episode reward for all methods. DQN learns effective rebalancing policies that outperform greedy heuristics at all tested scales.

Table 6: Dispatch benchmark results.

K	$\|\mathcal{S}\|$	DP Reward	DP Time	DQN Reward	Nearest	No-Rebal	Proportional
2	256	373.3	4.77	372.1 $\pm$ 1.1	361.0	367.0	357.7
3	4,096	—	—	543.6 $\pm$ 1.7	511.2	522.0	522.6
4	65,536	—	—	678.1 $\pm$ 3.2	600.4	610.2	649.2

The dispatch benchmark isolates the matching and allocation archetype common across ride-hailing, organ donation, and labor markets. The key insight is that myopic matching ignores the spatial externality of driver positioning; RL policies learn to sacrifice short-run matches for better long-run supply distribution across zones.

5.4 Benchmark 4: Pricing and Marketing

Dynamic discount targeting is a core problem in digital marketing: given a customer's interaction history, choose a discount level to maximize long-run revenue. The challenge is that discounts increase immediate conversion probability but may habituate customers to expect discounts, reducing future willingness to pay at full price.

Uber and Lyft use personalized promotions to manage rider retention and acquisition. Misra et al. (2019) demonstrated the value of incorporating microeconomic choice theory into scalable dynamic price experimentation. Their approach combines structural

demand estimation with RL-based policy optimization, showing that embedding economic theory into the reward and transition structure improves both sample efficiency and interpretability relative to purely model-free approaches.

We formalize a discount targeting MDP inspired by ride-sharing coupon optimization.

Definition D4 (Dynamic Discount Targeting MDP)**.** *The discount targeting MDP $\mathcal{M}_{\text{disc}}(n)$ is defined as follows, where n controls state complexity.*

1. *State components. Depending on n:*
   - $n \geq 1$*: recency* $\in \{0, \ldots, R-1\}$ *(days since last ride),*
   - $n \geq 2$*: frequency* $\in \{0, \ldots, F-1\}$ *(lifetime rides),*
   - $n \geq 3$*: last discount* $\in \{0, \ldots, D-1\}$ *(discount tier last given),*
   - $n \geq 4$*: habituation* $\in \{0, \ldots, H\}$ *(consecutive-discount counter),*
   - $n \geq 5$*: second-to-last discount* $\in \{0, \ldots, D-1\}$*.*

   With $R = 30$, $F = 20$, $D = 4$, $H = 5$, the state space at $n = 3$ is $30 \times 20 \times 4 = 2{,}400$ states.
2. *Action space. $\mathcal{A} = \{0\%, 10\%, 20\%, 30\%\}$, with $|\mathcal{A}| = 4$ discount tiers.*
3. *Conversion probability. Given state features, conversion probability follows a logistic model:*

$$p(\textit{convert}|s, a) = \sigma\left(\beta_0 - \beta_h \cdot h + \beta_d \cdot d_a + \beta_r \cdot r\right), \tag{42}$$

   where $\sigma(\cdot)$ is the sigmoid, h is habituation, d_a is the discount percentage, and r is recency.
4. *Reward function. Revenue conditional on conversion:*

$$r(s, a) = \begin{cases} R_{\text{base}} \cdot (1 - d_a) & \textit{if converted,} \\ 0 & \textit{otherwise.} \end{cases} \tag{43}$$

5. *Transitions. Conversion resets recency to 0 and increments frequency; non-conversion increments recency. Giving a discount increments habituation; not giving one decrements it.*
6. *Discount factor. $\gamma = 0.95$.*

The Bellman optimality equation incorporates the stochastic conversion:

$$V^*(s) = \max_{a \in \mathcal{A}} \left[\mathbb{E}\left[r(s, a) + \gamma\, V^*(s')\right]\right]. \tag{44}$$

The heuristic set contains three policies. The static ITE policy always offers 10% (a fixed personalization baseline). The threshold policy offers 30% if recency exceeds 20 days, else 0% (a recency-triggered rule). The no-discount policy never offers discounts.

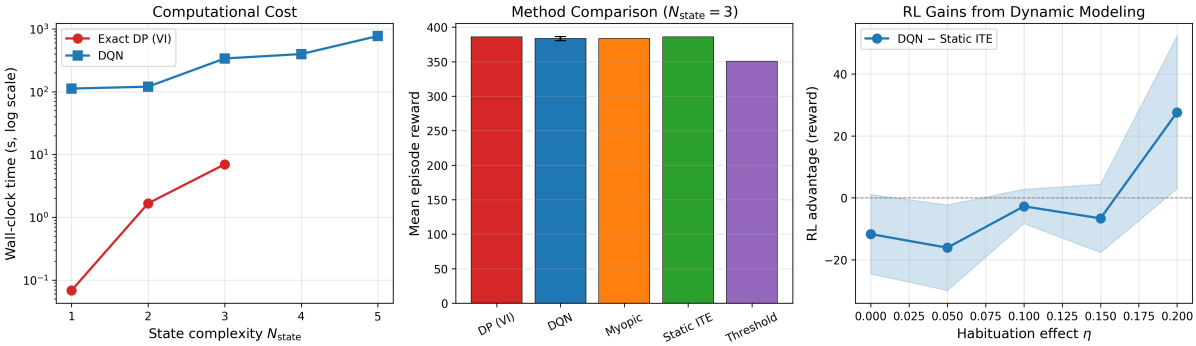

Figure 4: Discount targeting benchmark ($n = 3$). Learning curve showing DQN convergence to DP optimal, with heuristic baselines for comparison.

Table 7: Discount targeting benchmark results ($n = 3$).

N_{state}	$\lvert\mathcal{S}\rvert$	DP (VI)	DQN	Myopic	Static ITE	Threshold	DP Time (s)
1	30	390.0	380.6 ± 0.9	382.9	388.3	353.8	0.1
2	600	385.8	376.5 ± 1.5	382.9	388.3	353.8	1.7
3	2,400	385.8	383.5 ± 2.8	383.8	386.2	350.6	7.0
4	14,400	—	381.3 ± 3.5	383.8	386.2	350.6	—
5	57,600	—	386.8 ± 2.6	395.8	383.5	351.0	—

The discount targeting benchmark tests the pricing and personalization archetype. The key economic tension is between short-run conversion gains from discounting and long-run habituation costs. RL policies learn state-dependent discount schedules that account for the customer's discount history.

5.5 Benchmark 5: Revenue Management

Dynamic pricing requires setting prices that respond to demand conditions, inventory levels, and competitor behaviour. The sequential structure arises because today's price affects both current sales and the inventory available for future periods.

The classical foundation for dynamic pricing is the intensity control framework of Gallego and van Ryzin (1994), who showed that for perishable inventory with Poisson demand, the optimal price trajectory can be characterized analytically under specific demand models. Their results establish the theoretical benchmark: any RL-based pricing

system should match or exceed the Gallego-van Ryzin solution in settings where its assumptions hold, and demonstrate value in the richer settings where those assumptions break down.

Alibaba deployed RL for dynamic pricing on its AliExpress platform. Wang et al. (2019a) described a learning-based approach to real-time price adjustment, where the state space included current inventory levels, historical sales velocity, and competitor pricing signals, and actions were discrete price adjustments. The system was evaluated on real transaction data from AliExpress and demonstrated improvements in revenue relative to fixed-price and rule-based baselines.

Chen et al. (2023) applied RL to hotel revenue management, a domain where the classical approach relies on booking limit controls derived from dynamic programming under parametric demand models. Their formulation treats each hotel property as a finite-horizon MDP: the state encodes current occupancy by room type and remaining days until check-in, actions are accept/reject decisions for incoming reservation requests at posted prices, and the reward is cumulative revenue. The RL agent, trained on historical booking data, learned pricing policies that adapted to demand fluctuations more responsively than static booking-limit heuristics.

We formalize a hotel revenue management MDP that captures the essential pricing-under-scarcity structure of these deployments.

Definition D5 (Hotel Revenue Management MDP)**.** *The hotel revenue management MDP $\mathcal{M}_{\text{hotel}}(C)$ is defined as follows.*

1. *State space.* $\mathcal{S} = \{0, \ldots, T-1\} \times \{0, \ldots, C\} \times \{0, \ldots, D-1\}$*, where T is the booking horizon, C is the hotel capacity, and D is the number of demand intensity bins. A state (t, I, d) records days until check-in, remaining rooms, and recent demand intensity. The state space size is $|\mathcal{S}| = T \cdot (C+1) \cdot D$.*
2. *Action space.* $\mathcal{A} = \{p_1, \ldots, p_M\}$*, a set of $M = 5$ discrete price tiers.*
3. *Demand model. Each period, the number of potential arrivals follows* $\text{Poisson}(\lambda(d))$*, where $\lambda(d)$ depends on the demand intensity bin. Each arrival books with probability*

$$q(p) = \frac{1}{1 + \exp(\alpha \cdot (p - p_{\text{ref}}))}, \tag{45}$$

where $\alpha > 0$ is the price sensitivity and p_{ref} is the reference price.

4. *Reward function.* $r(s, a) = p_a \cdot$*bookings, where bookings* $= \min($*arrivals who book*, $I)$.
5. *Transitions. Inventory decrements by bookings. The demand intensity bin transitions stochastically based on realized demand. At* $t = 0$ *(check-in), the episode ends; unsold rooms yield zero revenue.*
6. *Discount factor.* $\gamma = 1.0$ *(finite horizon, no discounting).*

The Bellman equation for this finite-horizon problem is

$$V_t^*(I, d) = \max_{p \in \mathcal{A}} \left[\mathbb{E} \left[p \cdot B + V_{t-1}^*(I - B, d') \right] \right], \quad V_0^*(I, d) = 0, \tag{46}$$

where B is the random number of bookings and d' is the next demand intensity bin. The state space is $T(C + 1)D$, which grows linearly in C. With $T = 30$ and $D = 3$, the state space is $90(C + 1)$: approximately 990 states at $C = 10$ and 7,290 at $C = 80$. Although the growth is linear, exact DP requires computing the booking probability distribution for each state-action pair, making it slow for large C.

The heuristic set $\mathcal{H}$ contains three policies. The fixed price policy always charges the reference price p_{ref}, ignoring both inventory and time. The EMSR-b policy (Expected Marginal Seat Revenue) is a standard airline and hotel heuristic that raises prices when inventory is scarce relative to remaining time.[20] The myopic policy selects the price maximizing immediate expected revenue $p \cdot \lambda(d) \cdot q(p)$, ignoring future scarcity.

Table 8: Hotel revenue management benchmark results.

C	$\lvert\mathcal{S}\rvert$	DP Revenue	DP Time	DQN Revenue	Fixed	EMSR-b	Myopic
10	660	1662.6	54.51	1577.2 ± 20.6	1200.0	1308.8	1206.6
20	1,260	2415.4	115.74	2142.9 ± 70.7	2253.6	2081.2	2256.0
30	1,860	—	—	2712.6 ± 91.2	2932.8	2440.4	2865.0

The hotel benchmark tests the pricing archetype: choosing prices over a finite horizon with perishable inventory. The key economic tension is between selling early at low prices (guaranteed revenue) and waiting for higher-value customers (risky but potentially more profitable). RL policies learn state-dependent pricing schedules that simple heuristics cannot express.

[20] Our EMSR-b implementation uses a simplified rule: if $I/(C \cdot t/T) > 1.2$, price low; if < 0.5, price high; otherwise price at reference.

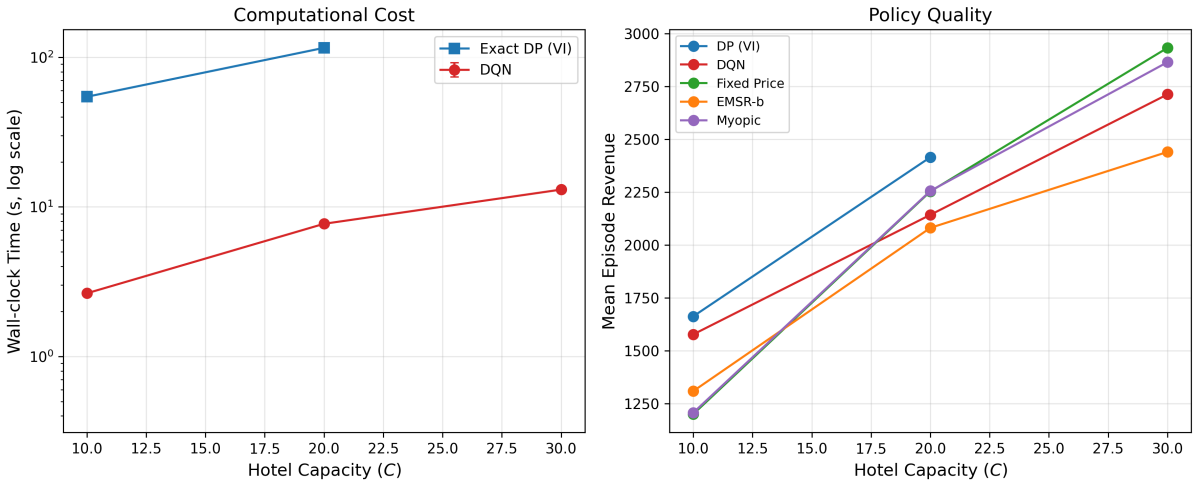

Figure 5: Hotel revenue management benchmark. Left: wall-clock time vs. capacity C. Right: mean episode revenue. DQN learns inventory-dependent pricing that outperforms the fixed-price and myopic baselines, approaching the EMSR-b heuristic and exceeding it at larger capacities.

5.6 Benchmark 6: Auctions and Advertising

Real-time bidding (RTB) is the dominant mechanism for online display advertising. When a user visits a webpage, an auction occurs in milliseconds: advertisers submit bids, the highest bidder wins the impression, and pays the second-highest bid (or a reserve price). An advertiser running a campaign must decide how aggressively to bid in each auction, subject to a total budget constraint over the campaign duration.

The budget pacing problem is to allocate a fixed budget across a sequence of auctions to maximize total conversions (or other objective). Bidding too aggressively early exhausts the budget before high-value impressions arrive later; bidding too conservatively leaves budget unspent. The optimal strategy depends on the distribution of auction clearing prices, the value of different impressions, and the remaining budget and time.

Wu et al. (2018) formalized RTB budget pacing as an MDP and demonstrated that RL-based bidding agents outperform rule-based pacing algorithms in simulated auctions calibrated to real RTB data. Their state representation included remaining budget, time elapsed, and recent win rates; actions were bid multipliers applied to a base bid. The RL agent learned to adapt bidding intensity to market conditions, increasing bids when competition was low and conserving budget when prices spiked.

We formalize an RTB budget pacing MDP.

Definition D6 (RTB Budget Pacing MDP)**.** *The RTB budget pacing MDP* $\mathcal{M}_{\mathrm{rtb}}(T)$ *is*

defined as follows.

1. *State space.* $\mathcal{S} = \{0, \ldots, T-1\} \times \{0, \ldots, B_{\text{bins}} - 1\} \times \{0, \ldots, W_{\text{bins}} - 1\}$*, encoding time period, discretized remaining budget, and discretized recent win rate. With* $T = 24$ *periods,* $B_{\text{bins}} = 10$ *budget bins, and* $W_{\text{bins}} = 3$ *win-rate bins, the state space has* $|\mathcal{S}| = 720$ *states.*
2. *Action space.* $\mathcal{A} = \{0, 0.5, 1.0, 1.5, 2.0\}$*, bid multipliers applied to a base bid.* $|\mathcal{A}| = 5$.
3. *Auction model. Each period, the agent participates in* $n = 20$ *second-price auctions. Clearing prices follow a log-normal distribution. The agent wins auctions where its bid exceeds the clearing price and pays the clearing price for each win.*
4. *Reward function. Conversions occur with probability* p_{conv} *per won impression:*

$$r(s, a) = \sum_{i=1}^{n} \mathbb{1}\{win_i\} \cdot \mathbb{1}\{convert_i\}. \tag{47}$$

5. *Transitions. Budget decrements by total spend. Win rate updates as an exponential moving average of period win rate. Campaign ends when* $t = T$ *or budget is exhausted.*
6. *Discount factor.* $\gamma = 1.0$ *(finite horizon, maximize total conversions).*

The Bellman equation is

$$V^*(t, b, w) = \max_{a \in \mathcal{A}} \left[\mathbb{E}\left[r(s, a) + V^*(t+1, b', w')\right]\right], \quad V^*(T, \cdot, \cdot) = 0. \tag{48}$$

The heuristic set contains three policies. The uniform pacing policy targets spending budget/remaining-periods per period. The greedy policy bids maximum until budget is exhausted. The PID controller adjusts bid multipliers based on the error between target and actual spend rate.

Table 9: RTB bidding benchmark results ($T = 24$).

T	$\lvert\mathcal{S}\rvert$	DP Conv.	DP Time	DQN Conv.	Uniform	Greedy	PID
12	360	10.0	2.52	9.9 ± 0.1	9.1	10.0	9.8
24	720	20.4	5.39	20.3 ± 0.4	14.1	20.4	19.3
48	1,440	—	—	31.2 ± 1.3	20.7	32.0	34.1

The RTB benchmark tests the budget allocation archetype under uncertainty. The

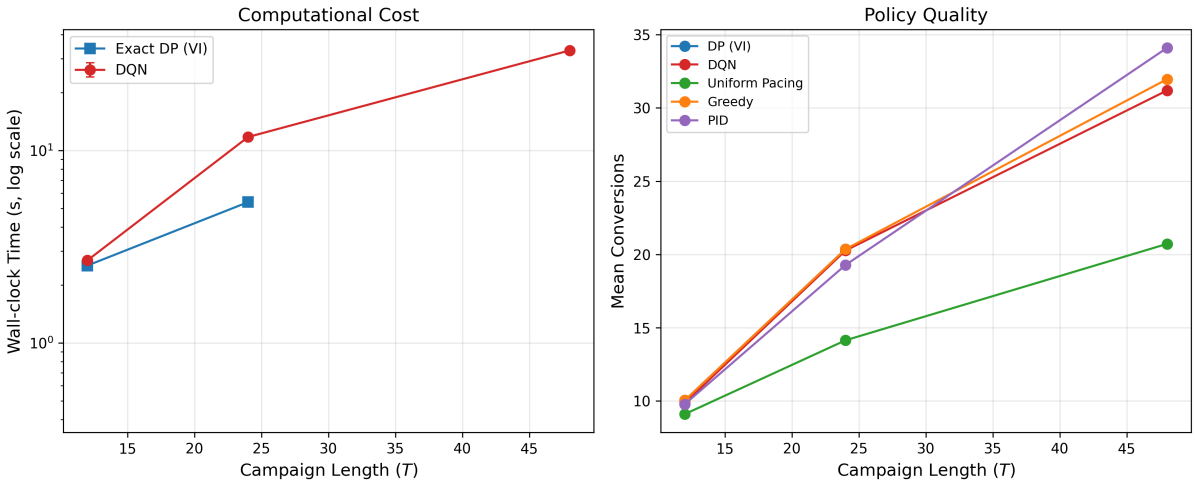

Figure 6: RTB bidding benchmark ($T = 24$). Learning curve showing DQN convergence. The greedy heuristic exhausts budget early; uniform pacing is conservative; RL learns adaptive bid adjustment.

key insight is that optimal pacing requires anticipating future auction conditions: if prices are currently high but expected to fall, the agent should conserve budget. RL policies learn this temporal arbitrage that rule-based pacers cannot.

5.7 Benchmark 7: Finance and Order Execution

Optimal execution of large equity orders is a central problem in algorithmic trading. A trader must liquidate a position of Q shares over T periods, balancing the tradeoff between trading quickly (to reduce exposure to price volatility) and trading slowly (to minimize market impact costs from moving the price against oneself).

The theoretical foundation is the Almgren-Chriss framework (Almgren and Chriss, 2001), which models execution as optimizing a mean-variance objective over trading trajectories. Under their assumptions of linear temporary and permanent impact, the optimal strategy is a deterministic schedule that can be computed analytically. However, real markets exhibit state-dependent liquidity: impact costs vary with trading volume, volatility regimes, and time of day.

Nevmyvaka et al. (2006) applied RL to optimal execution on real limit order book data, demonstrating that RL agents learned execution strategies that outperformed the Almgren-Chriss baseline by adapting to intraday liquidity patterns. JPMorgan's LOXM system reportedly uses RL for execution, learning when to trade aggressively versus passively based on market microstructure signals.

We formalize an order execution MDP.

Definition D7 (Order Execution MDP)**.** *The order execution MDP $\mathcal{M}_{\text{exec}}(T)$ is defined as follows.*

1. *State space.* $\mathcal{S} = \{0, \ldots, T-1\} \times \{0, \ldots, Q\} \times \{0, \ldots, V-1\} \times \{0, \ldots, \Sigma - 1\}$, *encoding time remaining, shares remaining, volume bin (current market volume), and volatility bin. With* $T = 20$, $Q = 10$, $V = 3$, $\Sigma = 3$, *the state space has* $|\mathcal{S}| = 1{,}980$ *states.*
2. *Action space.* $\mathcal{A} = \{0, 0.2, 0.4, 0.6, 0.8, 1.0\}$, *the fraction of remaining shares to trade this period.* $|\mathcal{A}| = 6$.
3. *Market impact model. Trading q shares incurs temporary impact (proportional to q^2) and permanent impact (proportional to q), scaled by current volume and volatility:*

$$c(q, v, \sigma) = \kappa_{\text{temp}} \cdot q^2 \cdot f(v) + \kappa_{\text{perm}} \cdot q \cdot f(v) + \kappa_{\text{risk}} \cdot q \cdot g(\sigma), \tag{49}$$

   where $f(v)$ is a volume scaling factor (lower volume $\Rightarrow$ higher impact) and $g(\sigma)$ is a volatility scaling factor.
4. *Reward function.* $r(s, a) = -c(q, v, \sigma)$, *the negative execution cost.*
5. *Transitions. Shares remaining decrements by q. Volume and volatility bins evolve as Markov chains. At $t = 0$, any unexecuted shares incur a large penalty (deadline miss).*
6. *Discount factor.* $\gamma = 1.0$ *(finite horizon, minimize total cost).*

The Bellman equation is

$$V^*(t, q, v, \sigma) = \max_{a \in \mathcal{A}} \left[r(s, a) + \mathbb{E}_{v', \sigma'} \left[V^*(t-1, q - \lfloor a \cdot q \rfloor, v', \sigma') \right] \right]. \tag{50}$$

The heuristic set contains three policies. The TWAP (Time-Weighted Average Price) policy trades q/t shares per period. The VWAP (Volume-Weighted Average Price) policy trades more in high-volume periods. The greedy policy executes all shares immediately.

The order execution benchmark tests the optimal timing archetype in finance. The key insight is that market conditions are informative: high volatility suggests trading faster to reduce risk; high volume suggests trading now because impact is low. RL policies learn these conditional strategies.

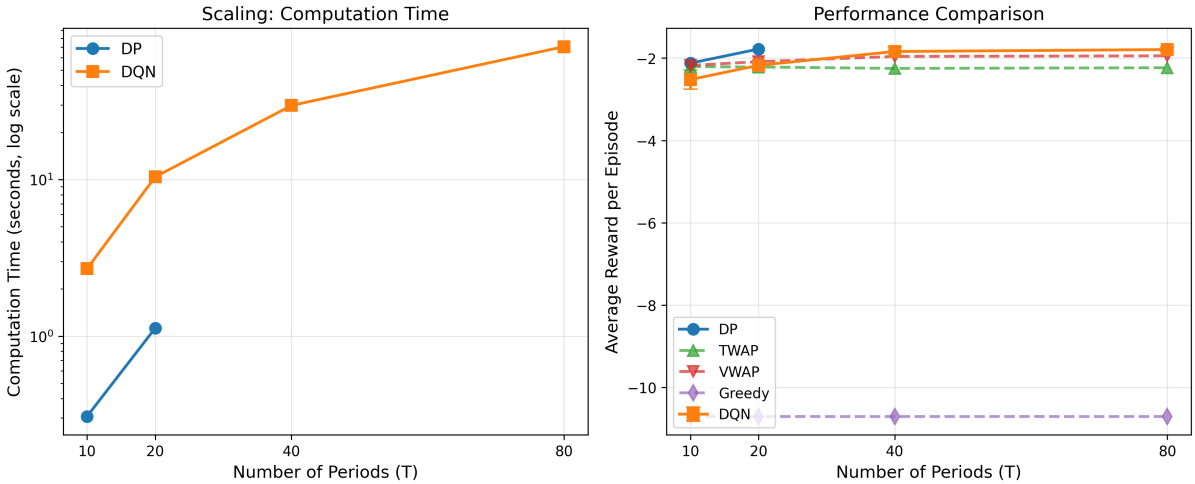

Figure 7: Order execution benchmark ($T = 20$). Learning curve showing DQN convergence. TWAP and VWAP are strong baselines; RL learns to deviate from uniform execution based on volatility and volume conditions.

Table 10: Order execution benchmark results ($T = 20$).

T	States	DP Time	DP Reward	DQN Time	DQN Reward	TWAP	VWAP
10	990	0.31	-2.12	2.70	-2.52	-2.21	-2.18
20	1980	1.12	-1.78	10.40	-2.18	-2.22	-2.09
40	3960	—	—	29.74	-1.84	-2.25	-1.96
80	7920	—	—	70.61	-1.79	-2.23	-1.94

5.8 Benchmark 8: Supply Chain and Inventory

Inventory management is among the oldest sequential decision problems in operations research. A firm must choose how much to order at each period, balancing the cost of holding excess stock against the cost of unmet demand, while facing uncertain future demand. In multi-echelon supply chains, this problem compounds: each stage's order becomes the next stage's demand, creating coupled dynamics across the network.

The classical theory begins with Clark and Scarf (1960), who established the optimal structure for two-echelon systems under specific cost and demand assumptions. Their echelon base-stock policy, where each stage orders up to a target level defined in terms of total downstream inventory, remains the analytical benchmark. Extensions to general networks, nonstationary demand, and capacity constraints have proven analytically intractable in most cases, motivating computational approaches.

Gijsbrechts et al. (2022) conducted a systematic evaluation of deep RL for inventory management across a range of supply chain configurations. Their study compared DQN and PPO agents against classical base-stock policies and found mixed results: RL matched or slightly exceeded classical methods in simple settings but struggled with multi-echelon problems where the state space grows combinatorially with the number of stages.

We formalize a serial multi-echelon inventory system as our benchmark MDP.

Definition D8 (Multi-Echelon Inventory MDP)**.** *The serial multi-echelon inventory MDP $\mathcal{M}_{\text{inv}}(K)$ is defined as follows.*

1. *State space. $\mathcal{S} = \{0, \ldots, I_{\max}\}^K$, where $I_{\max}$ is the maximum inventory per echelon and K is the number of echelons. A state $s = (I_1, \ldots, I_K)$ records on-hand inventory at each echelon. The state space size is $|\mathcal{S}| = (I_{\max} + 1)^K$.*
2. *Action space. $\mathcal{A} = \{0, 1, \ldots, Q_{\max}\}$, the order quantity for echelon 1 (the most downstream echelon facing customer demand). $|\mathcal{A}| = Q_{\max} + 1$.*
3. *Demand. Customer demand arrives at echelon 1 as $D \sim \text{Poisson}(\lambda)$.*
4. *Dynamics. Each period: (i) demand D is realized; (ii) echelon 1 fulfills $\min(I_1, D)$ units; (iii) unfilled demand is backlogged; (iv) each upstream echelon $k > 1$ ships $\min(I_k, \textit{deficit}_{k-1})$ downstream; (v) the order of q units arrives at echelon K from an external supplier with infinite capacity (lead time $L = 1$).*

5. *Cost function. The per-period cost is*

$$c(s, a, D) = h \sum_{k=1}^{K} \max(I'_k, 0) + b \cdot \max(0, D - I_1) + c_o \cdot q, \tag{51}$$

where h is the per-unit holding cost, b is the per-unit backorder cost, c_o is the per-unit ordering cost, and I'_k is the post-shipment inventory.

6. *Reward. $r(s, a) = -\mathbb{E}_D[c(s, a, D)]$ (negative expected cost).*
7. *Discount factor. $\gamma = 0.95$.*

The Bellman optimality equation is

$$V^*(s) = \max_{q \in \mathcal{A}} \left[\mathbb{E}_D \left[-c(s, q, D) + \gamma V^*(s') \right] \right], \tag{52}$$

where s' is the post-shipment, post-demand inventory vector. The state space $(I_{\max}+1)^K$ grows exponentially in K: with $I_{\max} = 15$, it is 256 at $K = 2$, 4,096 at $K = 3$, 65,536 at $K = 4$, and over 10^6 at $K = 5$. Value Iteration is feasible for $K \leq 3$ and becomes impractical beyond.

The heuristic set $\mathcal{H}$ contains three policies. The base-stock policy orders up to a target level $S^* = F_D^{-1}(b/(b+h))$ whenever inventory at echelon 1 falls below S^*, where F_D is the demand CDF. This is the newsvendor solution applied independently to echelon 1, ignoring upstream coupling. The (s, S) policy places an order of $S - I_1$ units whenever $I_1 < s$, with s and S set by rule of thumb. The fixed order policy orders $\lceil \lambda \rceil$ units every period regardless of state, a naive replenishment rule.

Table 11: Multi-echelon inventory benchmark results.

K	States	DP Time	DP Reward	DQN Time	DQN Reward	Base-stock	(s,S)
2	81	7.55	-358.18	4.50	-368.87	-458.98	-562.76
3	729	—	—	9.81	-466.74	-544.84	-586.60
4	6561	—	—	14.06	-508.51	-585.52	-636.53

The inventory benchmark tests the quantity choice archetype. The multi-echelon structure introduces coupling between echelons that single-echelon heuristics ignore. The base-stock policy is provably optimal for the single-echelon newsvendor problem, but extending it independently to each echelon in a serial system is suboptimal: it fails to account for how upstream inventory affects downstream replenishment lead times. RL policies learn coordinated ordering strategies that exploit the full state vector.

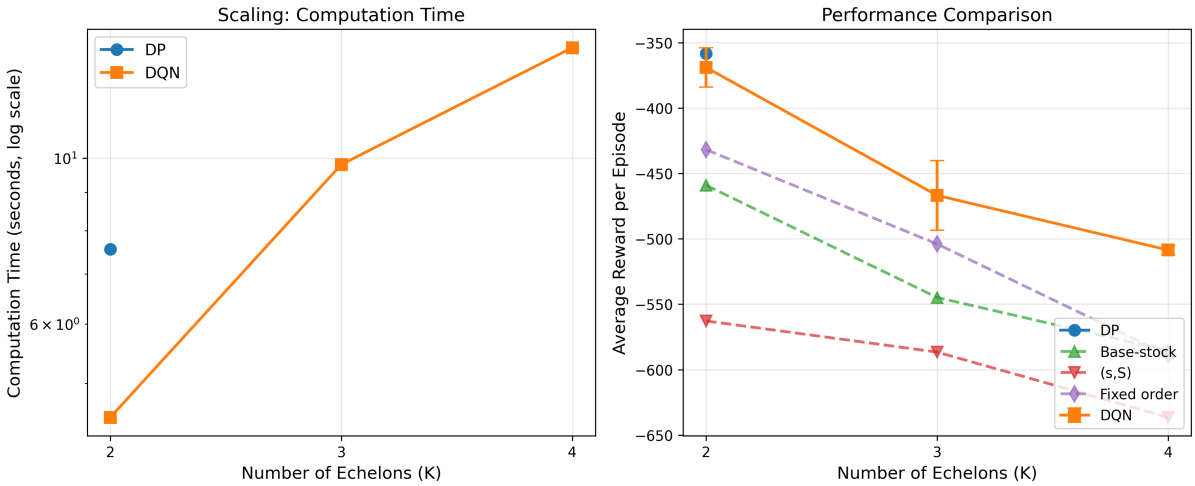

Figure 8: Multi-echelon inventory benchmark. Left: wall-clock time vs. number of echelons K. Right: mean episode reward (negative cost). The base-stock heuristic, optimal for a single echelon, degrades as the supply chain lengthens; RL learns coordinated ordering that accounts for upstream inventory.

5.9 Benchmark 9: Energy and Operations

Operational control problems, where a system must be continuously adjusted to maintain performance under changing conditions, are natural MDPs. The state is the current system configuration and environmental conditions; actions are control adjustments; rewards reflect operational cost or efficiency.

Google applied machine learning to data center cooling, autonomously controlling fans, chillers, and other equipment to maintain safe temperatures while minimizing energy usage. Gao (2014) reported that the initial system, based on neural network predictions of Power Usage Effectiveness (PUE), reduced cooling energy consumption by up to 40%, yielding a 15% reduction in overall PUE. Lazic et al. (2018) subsequently formalized the problem as a model-predictive control task and deployed a safe RL approach that further improved cooling efficiency while providing formal safety guarantees against temperature constraint violations. The state space included over 100 sensor readings (temperatures, pressures, flow rates); actions were setpoints for cooling equipment. The system operated continuously in production, adapting to seasonal and load-driven variations without manual intervention.

We formalize a multi-zone data center cooling problem as our benchmark MDP.

Definition D9 (Data Center Cooling MDP)**.** *The data center cooling MDP* $\mathcal{M}_{\mathrm{dc}}(N)$ *is*

defined as follows.

1. *State space.* $\mathcal{S} = \{0, \ldots, T_{\text{bins}} - 1\}^N \times \{0, \ldots, L_{\text{bins}} - 1\}^N \times \{0, \ldots, E_{\text{bins}} - 1\}$*, where* T_{bins} *is the number of temperature bins per zone,* L_{bins} *is the number of server load bins per zone,* E_{bins} *is the number of external temperature bins, and* N *is the number of zones. The state space size is* $|\mathcal{S}| = T_{\text{bins}}^N \cdot L_{\text{bins}}^N \cdot E_{\text{bins}}$.
2. *Action space.* $\mathcal{A} = \{c_1, \ldots, c_M\}^N$*, where each* c_j *is a discrete cooling intensity level. The joint action space size is* M^N*, exponential in the number of zones.*
3. *Thermal dynamics. For each zone* k*, the next-period temperature evolves as*

$$T_{k,t+1} = T_{k,t} + \alpha \cdot L_{k,t} - \beta \cdot c_{k,t} + \delta \cdot E_t + \varepsilon_{k,t}, \tag{53}$$

   where α *is the heat gain from server load,* β *is the cooling effectiveness,* δ *is the external temperature influence, and* $\varepsilon_{k,t}$ *is i.i.d. noise.*
4. *Cost function. The per-period cost is*

$$c(s, a) = \sum_{k=1}^{N} \left[\kappa_1 c_k + \kappa_2 c_k^2 + \Gamma \cdot \mathbb{1}\{T_k \geq \bar{T}\} \cdot (T_k - \bar{T} + 1) + \xi |T_k - T^*| \right], \tag{54}$$

   where κ_1, κ_2 *are energy cost coefficients (linear and quadratic),* Γ *is the overheat penalty,* $\bar{T}$ *is the overheat threshold,* T^* *is the target temperature, and* ξ *is the comfort penalty coefficient.*
5. *Reward function.* $r(s, a) = -c(s, a)$.
6. *Transitions. Temperature evolves per* (53) *and is discretized. Server load and external temperature follow independent Markov chains.*
7. *Discount factor.* $\gamma = 0.95$.

The Bellman optimality equation is

$$V^*(s) = \max_{a \in \mathcal{A}} \left[r(s, a) + \gamma \sum_{s'} P(s'|s, a) \, V^*(s') \right]. \tag{55}$$

The state space grows as $T_{\text{bins}}^N \cdot L_{\text{bins}}^N \cdot E_{\text{bins}}$, which is exponential in N. With $T_{\text{bins}} = 6$, $L_{\text{bins}} = 4$, and $E_{\text{bins}} = 3$, a single zone has $6 \cdot 4 \cdot 3 = 72$ states; two zones have $6^2 \cdot 4^2 \cdot 3 =$ 1,728 states; four zones exceed 5×10^6 states. The action space also grows exponentially: with 4 cooling levels per zone, the joint action space is 4^N, reaching 256 at $N = 4$. Value Iteration is feasible for $N = 1$ and infeasible beyond.

The heuristic set $\mathcal{H}$ contains three policies. The fixed setpoint policy applies medium cooling intensity ($c = 0.5$) to all zones regardless of conditions. The load-proportional policy sets cooling intensity proportional to server load at each zone: high load triggers high cooling, idle servers receive minimal cooling. The PID controller adjusts cooling based on the temperature error from target, with proportional and integral terms that smooth the response over time.

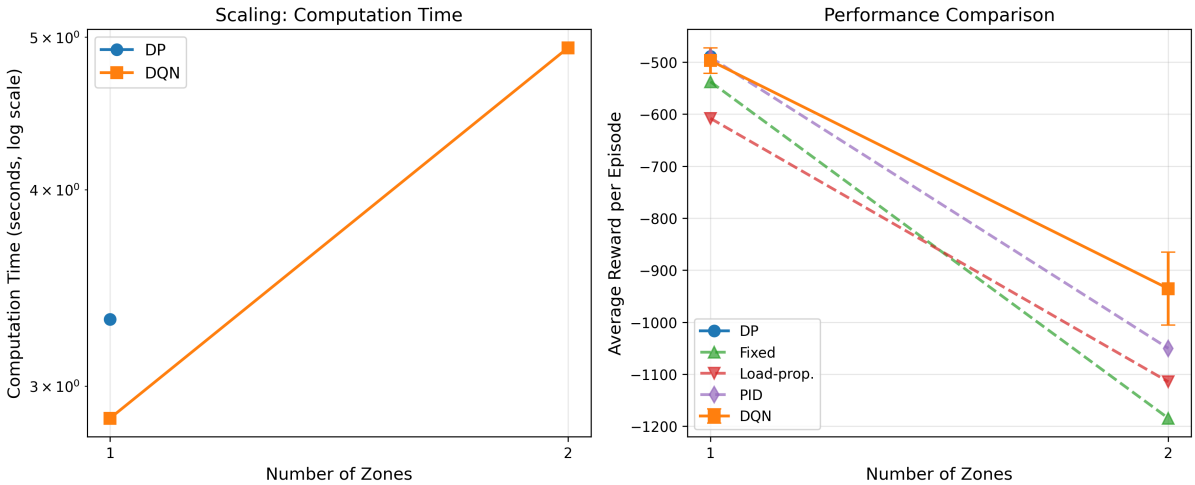

Figure 9: Data center cooling benchmark. Left: wall-clock time vs. number of zones N. Right: mean episode reward (negative operating cost). The fixed setpoint wastes energy on idle zones. Load-proportional cooling improves efficiency but ignores thermal dynamics. RL learns zone-aware cooling schedules that balance energy cost against overheating risk.

Table 12: Data center cooling benchmark results.

Zones	States	DP Time	DP Reward	DQN Time	DQN Reward	Fixed	PID
1	72	3.31	-489.18	2.86	-497.05	-537.58	-490.82
2	1728	—	—	4.92	-935.30	-1184.42	-1050.10

The datacenter benchmark tests the continuous control archetype. The multi-zone structure creates coupling between zones (through external temperature and shared cooling infrastructure) that single-zone heuristics ignore. The PID controller adapts to temperature deviations but does not anticipate load changes; RL policies learn predictive cooling strategies that pre-emptively adjust cooling before temperatures reach critical thresholds.

5.10 Summary of Benchmark Suite

Table 13 summarizes the nine benchmarks, their economic archetypes, complexity dials, state space ranges, and the principal lesson each teaches about the relationship between RL and classical methods.

Table 13: Summary of the nine economic benchmarks.

#	Benchmark	Archetype	Dial	$\lvert\mathcal{S}\rvert$ Range	Lesson
Pedagogical Examples					
1	Gridworld	Navigation	N (grid size)	100–400	Algorithm validation
2	Bus Engine	Replacement	N (engines)	1,296–46,656	Structural estimation
Production Applications					
3	Dispatch	Matching	K (zones)	$1{,}296$–60×10^6	Spatial externalities
4	Discount	Personalization	n (features)	2,400–72,000	Habituation dynamics
5	Hotel RM	Pricing	C (capacity)	990–7,290	Perishable inventory
6	RTB	Budget pacing	T (periods)	720–2,160	Temporal arbitrage
7	Execution	Optimal timing	T (periods)	1,980–5,940	Market microstructure
8	Inventory	Quantity choice	K (echelons)	256–10^6	Supply chain coupling
9	Datacenter	Control	N (zones)	72–5×10^6	Multi-zone coupling

Taken together, the nine benchmarks cover major economic decision archetypes: navigation and validation (gridworld), durable goods replacement (bus engine), spatial allocation (transport dispatch), dynamic personalization (discount targeting), pricing with perishable inventory (hotel), budget allocation in auctions (RTB), optimal execution timing (finance), multi-echelon quantity choice (inventory), and operational control (data center cooling). Each benchmark confirms the two-tier evaluation structure: RL matches DP where DP is feasible (Tier 1 credibility) and outperforms practitioner heuristics where DP is not (Tier 2 value).

5.11 Common Patterns and Open Challenges

Several patterns emerge across the nine benchmarks and the deployed systems they represent. The most successful applications share a common structure: high-frequency decisions, abundant logged data for offline pre-training, and a well-defined scalar reward. Nearly all production systems add safety mechanisms such as action clipping, fallback to rule-based policies, or conservative offline training before online deployment. The resulting gains are often modest in percentage terms (0.5–2% for dispatch, 15% for trade execution) but economically large due to the scale of operations.

Significant challenges remain. Reward misalignment can produce policies that optimize a metric while degrading the economic outcome. Multi-objective settings require reward engineering that current practice handles through ad hoc weighted sums. Sample efficiency is a bottleneck where experimentation is costly or risky. Interpretability of RL policies lags behind traditional approaches, limiting adoption in regulated industries.

6 Solving Economic Models with Reinforcement Learning

This chapter reviews papers that use reinforcement learning algorithms to solve or estimate structural economic models. The common thread is the use of a genuine RL training loop: Q-learning, temporal-difference learning, policy gradient, or actor-critic methods applied to economic environments where dynamic programming is either infeasible or impractical. We exclude papers that use neural network function approximation without an RL training mechanism. Inverse reinforcement learning is treated in the sister survey (Rust and Rawat, 2026a).

Throughout this chapter we adopt a unified notation. An MDP is a tuple $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$ where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $P(s'|s,a)$ is the transition kernel, $r(s,a)$ is the per-period reward, and $\gamma \in [0,1)$ is the discount factor.[21] A policy $\pi : \mathcal{S} \to \Delta(\mathcal{A})$ maps states to distributions over actions. The value function under π is $V^\pi(s) = \mathbb{E}_\pi\left[\sum_{t=0}^\infty \gamma^t r(s_t, a_t) \mid s_0 = s\right]$, and the action-value function is $Q^\pi(s,a) = r(s,a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s,a)}[V^\pi(s')]$. The optimal value function satisfies $V^*(s) = \max_{a \in \mathcal{A}} Q^*(s,a)$.

6.1 Dynamic Discrete Choice Estimation

Dynamic discrete choice (DDC) models are the workhorse of structural empirical economics. An agent in state $s \in \mathcal{S}$ chooses action $a \in \mathcal{A}$ (where $|\mathcal{A}|$ is finite) to maximize

$$\mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t \left(u(s_t, a_t; \theta) + \varepsilon_t(a_t)\right)\right], \tag{56}$$

where $u(s,a;\theta) = z(s,a)^\top \theta$ is the deterministic component of per-period utility, $\theta \in \mathbb{R}^d$ collects the structural parameters, and $\varepsilon_t(a)$ is an idiosyncratic shock, typically drawn

[21] Several of the papers reviewed here use β for the discount factor, following economics convention. We translate all results to γ for consistency with the RL literature and the rest of this survey.

i.i.d. from a Type I extreme value distribution.[22] The econometrician observes panel data $\{(s_{it}, a_{it})\}_{i=1,\dots,n}^{t=1,\dots,T}$ and seeks to recover θ.

The classical approach of Rust (1987) requires repeated solution of a fixed-point problem (nested fixed-point, or NFXP),[23] while the conditional choice probability (CCP) method of Hotz and Miller (1993) avoids this by exploiting an invertibility result linking choice probabilities to value function differences. Both methods face severe computational challenges when the state space is continuous or high-dimensional: NFXP requires discretization and repeated Bellman iteration, while CCP requires nonparametric estimation of transition densities.

These computational bottlenecks motivate bringing RL algorithms directly into the DDC estimation pipeline. Two recent papers pursue this approach.

6.1.1 Adusumilli and Eckardt (2022): TD Learning for CCP Estimation

Adusumilli et al. (2022) adapt temporal-difference (TD) learning to estimate the recursive terms that arise in CCP-based estimation, entirely avoiding specification or estimation of transition densities.

The CCP approach requires computing two functions $h : \mathcal{A} \times \mathcal{S} \to \mathbb{R}^d$ and $g : \mathcal{A} \times \mathcal{S} \to \mathbb{R}$ that solve the recursive equations

$$h(a, s) = z(s, a) + \gamma\, \mathbb{E}[h(a', s') \mid a, s], \tag{57}$$

$$g(a, s) = e(a, s) + \gamma\, \mathbb{E}[g(a', s') \mid a, s], \tag{58}$$

where $e(a, s) = \gamma_{\mathrm{E}} - \ln P(a|s)$ under logit errors (γ_{E} denoting the Euler constant), and the expectation is over the next-period state-action pair (s', a') given the transition kernel $P(s'|s, a)$ and the observed policy $P(a|s)$. Both h and g have a value-function structure: they satisfy a Bellman-like recursion under the observed (data-generating) policy, not the optimal policy.

This observation is the key insight. Standard TD learning estimates value functions under a fixed policy from trajectory data, which is exactly what is needed here.

[22] The Type I extreme value (Gumbel) distribution has CDF $F(\varepsilon) = \exp(-\exp(-\varepsilon))$. It yields the logit choice probability $P(a|s) = \exp(V(s, a))/\sum_{a'} \exp(V(s, a'))$, making the model tractable.

[23] NFXP solves the likelihood maximization by nesting a Bellman fixed-point computation inside each evaluation of the likelihood function. Each likelihood evaluation requires solving the entire dynamic programming problem, making NFXP computationally expensive.

Adusumilli et al. (2022) propose two methods.

The core idea is to approximate h and g with linear combinations of basis functions, then use regression on observed state-action sequences to estimate the weights. This reduces the infinite-dimensional fixed-point problem to a finite-dimensional linear system.

The first is the linear semi-gradient method.[24] Approximate $h(a, s) \approx \phi(a, s)^\top w$ where $\phi : \mathcal{A} \times \mathcal{S} \to \mathbb{R}^p$ is a vector of basis functions (e.g., polynomials in state variables) and $w \in \mathbb{R}^p$ are the weights to be estimated. The TD(0) fixed-point equation is

$$\mathbb{E}\left[\phi(a, s)\left(\phi(a, s) - \gamma\phi(a', s')\right)^\top\right] w = \mathbb{E}\left[\phi(a, s)\, z(s, a)\right]. \tag{59}$$

The sample analog replaces population expectations with averages over the observed panel:

$$\hat{w} = \left(\frac{1}{n(T-1)} \sum_{i=1}^{n} \sum_{t=1}^{T-1} \phi_{it}\left(\phi_{it} - \gamma\phi_{i,t+1}\right)^\top\right)^{-1} \left(\frac{1}{n(T-1)} \sum_{i=1}^{n} \sum_{t=1}^{T-1} \phi_{it}\, z_{it}\right), \tag{60}$$

where $\phi_{it} = \phi(a_{it}, s_{it})$ and $z_{it} = z(s_{it}, a_{it})$. This requires inverting a $p \times p$ matrix, where p is the number of basis functions, making computation trivial in most settings. No transition density estimation is needed: the method uses only observed sequences of current and next-period state-action pairs.

The second method is approximate value iteration (AVI). Rather than fixing a linear basis, AVI constructs a sequence of approximations $\hat{h}^{(k)}$ by iterating the Bellman-like operator and solving a nonparametric regression at each step. At iteration k, given the current approximation $\hat{h}^{(k-1)}$, one constructs pseudo-outcomes

$$Y_{it}^{(k)} = z_{it} + \gamma\, \hat{h}^{(k-1)}(a_{i,t+1}, s_{i,t+1}) \tag{61}$$

and then fits $\hat{h}^{(k)}$ by regressing $Y_{it}^{(k)}$ on (a_{it}, s_{it}) using any machine learning method: LASSO, random forests, or neural networks. This is the first DDC estimator compatible with arbitrary ML prediction methods, enabling application to very high-dimensional state spaces.

With $\hat{h}$ and $\hat{g}$ in hand, structural parameters are recovered by pseudo-maximum like-

[24] The method is called "semi-gradient" because it computes the gradient of only the prediction $\phi(a, s)^\top w$ with respect to w, not the full TD error including the bootstrap target $\phi(a', s')^\top w$. This avoids differentiating through the target but sacrifices guaranteed convergence in some settings.

lihood estimation (PMLE). For continuous state spaces, Adusumilli et al. (2022) derive a locally robust correction to the PMLE criterion that accounts for the nonparametric first-stage estimation of value terms, restoring $\sqrt{n}$-convergence of $\hat{\theta}$.[25]

Theorem 2 (Adusumilli et al. (2022), Theorem 1)**.** *Under regularity conditions, the linear semi-gradient estimator $\hat{h}$ satisfies $\|\hat{h} - h\|_2 = O_P(n^{-1/2}(T-1)^{-1/2})$, where $\|\cdot\|_2$ denotes the $L^2(P)$ norm.*[26]

Theorem 3 (Adusumilli et al. (2022), Theorem 3)**.** *Under regularity conditions on the ML method used in AVI, the locally robust PMLE estimator $\hat{\theta}_{LR}$ satisfies $\sqrt{n}(\hat{\theta}_{LR} - \theta^*) \xrightarrow{d} \mathcal{N}(0, \Sigma)$ for an explicit variance Σ, even when the state space is continuous.*

Monte Carlo experiments on a dynamic firm entry model with seven structural parameters and five continuous state variables show that the TD-based estimators achieve a 4- to 11-fold reduction in mean squared error compared to CCP estimators using state-space discretization.

For dynamic discrete games, the method extends naturally. Standard CCP-based estimation of games requires integrating out other players' actions, which becomes intractable with many players or continuous states. TD learning avoids this entirely, since it works directly with the joint empirical distribution of states and their successors. The "integrating out" is done implicitly within sample expectations.

The limitation is the linear semi-gradient's dependence on the choice of basis functions, though cross-validation can guide this selection. AVI with flexible ML methods relaxes this constraint at the cost of slightly higher small-sample bias, which the locally robust correction addresses.

6.1.2 Hu and Yang (2025): Policy Gradient for DDC Estimation

Hu and Yang (2025) take a different approach: rather than using TD learning within the CCP framework, they use policy gradient methods to solve the inner-loop dynamic programming problem that arises in simulation-based estimation.

[25] An estimator has $\sqrt{n}$-convergence if its error shrinks at rate $n^{-1/2}$ where n is the sample size. This is the standard parametric rate; slower rates (e.g., $n^{-1/4}$) indicate efficiency loss from nonparametric first stages.

[26] The $L^2(P)$ norm is $\|f\|_2 = (\int f(x)^2 \, dP(x))^{1/2}$, measuring average squared deviation under the probability measure P. This is the natural norm for mean-squared-error analysis.

The estimation framework is indirect inference.[27] Define a vector of data moments $\mathbf{M}_d$ computed from the observed panel. For candidate parameter θ, simulate the model under θ to produce simulated moments $\mathbf{M}_s(\theta)$. The estimator minimizes the quadratic distance:

$$\hat{\theta} = \arg\min_{\theta} \left(\mathbf{M}_d - \mathbf{M}_s(\theta)\right)^\top \mathbf{W} \left(\mathbf{M}_d - \mathbf{M}_s(\theta)\right), \tag{62}$$

where $\mathbf{W}$ is a positive definite weight matrix. Computing $\mathbf{M}_s(\theta)$ requires solving for the optimal policy under θ, which is the inner-loop problem.

The inner loop parameterizes the policy directly as a neural network $\pi_\phi(a|s,\varepsilon)$, where ϕ denotes the network weights and ε captures unobserved state components. The policy takes the softmax form

$$\pi_\phi(a|s,\varepsilon) = \frac{\exp\left(f_\phi(s,\varepsilon,a)\right)}{\sum_{a'\in\mathcal{A}} \exp\left(f_\phi(s,\varepsilon,a')\right)}, \tag{63}$$

where $f_\phi : \mathcal{S} \times \mathbb{R}^k \times \mathcal{A} \to \mathbb{R}$ is a feedforward neural network. The policy is optimized by REINFORCE-style gradient ascent:

$$\nabla_\phi J(\phi) = \mathbb{E}_{\tau\sim\pi_\phi}\left[\sum_{t=0}^{T} \nabla_\phi \log \pi_\phi(a_t|s_t)\, Q^{\pi_\phi}(s_t,a_t)\right], \tag{64}$$

where $\tau = (s_0, a_0, s_1, a_1, \ldots)$ is a trajectory sampled under π_ϕ and Q^{π_ϕ} is the action-value function under the current policy. In practice, $Q^{\pi_\phi}(s_t,a_t)$ is replaced by the empirical return $\sum_{k=0}^{T-t} \gamma^k r_{t+k}$ or a learned baseline.

The key advantage is that policy gradient methods handle continuous state transitions naturally. The NFXP approach requires either discretizing $\mathcal{S}$ or numerically integrating over $P(s'|s,a)$ at every Bellman iteration. Policy gradient sidesteps both: gradients flow through sampled trajectories, and the transition kernel is used only implicitly through simulation. This makes the method applicable to models with continuous state variables, high-dimensional unobserved heterogeneity, and complex transition dynamics.

The estimation proceeds as a nested loop. For each candidate θ in the outer minimization (62), the inner loop runs policy gradient until convergence, producing an approximately optimal policy $\pi_{\phi^*(\theta)}$. This policy is then used to simulate data and compute $\mathbf{M}_s(\theta)$. The outer loop updates θ toward the minimizer of the moment distance.

[27] Indirect inference (Gourieroux et al., 1993) estimates structural parameters by matching moments from simulated data to moments from observed data, avoiding direct evaluation of the likelihood function.

The main limitation is computational cost: each outer-loop evaluation requires a full policy gradient optimization in the inner loop. The authors report that careful warm-starting (initializing ϕ at the previous inner-loop solution when θ changes incrementally) substantially reduces this burden.

6.2 Dynamic Oligopoly and Strategic Interaction

Dynamic oligopoly models combine elements of game theory and dynamic programming: firms choose actions (prices, quantities, investment levels) strategically, anticipating both their own future states and the strategies of competitors. The state space grows combinatorially in the number of firms, and the requirement to compute best responses against other firms' strategies adds further computational burden.

6.2.1 Asker, Fershtman, Jeon, and Pakes (2020): Q-Learning in Dynamic Procurement Auctions

Asker et al. (2020) develop a computational framework for analyzing dynamic procurement auctions with serially correlated asymmetric information. Their approach builds on the Experience-Based Equilibrium (EBE) concept of Fershtman and Pakes (2012), which computes equilibria by simulating industry trajectories and updating strategies toward best responses. EBE evaluates values only on recurrently visited states rather than the full state space, making it feasible for large state spaces. While Fershtman and Pakes (2012) use iterative value estimation without a standard RL algorithm, Asker et al. (2020) add explicit Q-learning updates.

The model is a repeated first-price sealed-bid auction with two firms. Each firm i maintains a private inventory state $\omega_{i,t}$ (stock of unharvested timber, in their application). The state evolves endogenously: winning an auction increases inventory, while harvesting depletes it. Each firm's private state is not observed by its competitor except at periodic revelation events.

The key computational innovation is the use of Q-learning to compute equilibrium strategies. Each firm i maintains a Q-function $Q_i : \mathcal{S}_i \times \mathcal{A}_i \to \mathbb{R}$, where $\mathcal{S}_i$ encodes firm i's information set (its own inventory, beliefs about the competitor's inventory, public history) and $\mathcal{A}_i$ is its action set (participation decision and bid level). The Q-function

satisfies

$$Q_i(s,a) = \mathbb{E}\left[r_i(s,a,a_{-i}) + \gamma \max_{a' \in \mathcal{A}_i} Q_i(s',a') \;\middle|\; s,a \right], \tag{65}$$

where $r_i(s,a,a_{-i})$ is firm i's per-period profit given the state, its own action a, and the competitor's action a_{-i}, and the expectation is taken over the competitor's strategy and the stochastic transitions.

Firms update Q-values using tabular Q-learning:

$$Q_i(s,a) \leftarrow Q_i(s,a) + \alpha \left[r_i + \gamma \max_{a' \in \mathcal{A}_i} Q_i(s',a') - Q_i(s,a) \right], \tag{66}$$

where $\alpha > 0$ is the learning rate. The equilibrium computation iterates: firms simultaneously update their Q-functions based on simulated play against each other's current strategies. Strategies are derived from Q-values using an ε-greedy rule or Boltzmann exploration.[28]

Asker et al. (2020) add a boundary consistency condition to the EBE concept that restricts behavior at the boundary of the recurrent state class, reducing the multiplicity of equilibria. Their numerical analysis reveals that information sharing between firms can, through increased precision of beliefs about competitor states, induce firms to spend more time in states where competition is less intense. The dynamic RL-computed equilibrium yields qualitatively different predictions from both static analysis and myopic ($\gamma = 0$) benchmarks: with dynamics, information sharing decreases average bids and increases average profits, while the myopic benchmark shows negligible effects.

The limitation of this approach is the tabular representation: the Q-function is stored as a lookup table over discretized states and actions, restricting applicability to models with moderate state-space dimension.

6.2.2 Hollenbeck (2019): TD Learning for Merger Analysis with Innovation

Hollenbeck (2019) uses RL to solve a dynamic oligopoly model with endogenous mergers, entry, exit, and quality investment. The model extends the Ericson-Pakes framework to study how horizontal mergers affect innovation incentives.

The industry state is $\Omega = (\omega_1, \ldots, \omega_n)$ where $\omega_i \in \{1, \ldots, \omega_{\max}\}$ is firm i's product

[28] ε-greedy selects the greedy action $argmax_a\, Q(s,a)$ with probability $1-\varepsilon$ and a uniformly random action otherwise. Boltzmann exploration selects action a with probability $\propto \exp(Q(s,a)/\tau)$ where $\tau > 0$ is a temperature parameter.

quality. Firms produce differentiated goods and compete in prices (Bertrand competition with logit demand). In each period, firms simultaneously choose investment levels, entry/exit decisions, and potentially initiate merger negotiations.

Each firm i computes its continuation value $V_i(\Omega)$ from the industry state. Because the state space is the product of all firms' quality levels plus industry structure (number of active firms, recent mergers), exact dynamic programming is infeasible for industries with more than two or three firms. Hollenbeck (2019) instead uses temporal-difference learning to estimate values from simulated industry trajectories.

The TD(0) update for firm i's value function is

$$V_i(\Omega) \leftarrow V_i(\Omega) + \alpha \left[\Pi_i(\Omega, \mathbf{a}) + \gamma V_i(\Omega') - V_i(\Omega)\right], \tag{67}$$

where $\Pi_i(\Omega, \mathbf{a})$ denotes firm i's per-period profit given industry state Ω and the joint action profile $\mathbf{a}$ (investment, entry/exit, merger decisions),[29] γ is the discount factor, and Ω' is the realized next-period state. The algorithm uses ε-greedy exploration to prevent convergence to locally suboptimal equilibria.

The equilibrium computation follows the Pakes-McGuire iterative scheme:[30] simulate long industry histories, update each firm's value function via (67), re-derive best-response strategies from updated values, and repeat. The RL component replaces the exact Bellman iteration step that would be infeasible over the full state space, focusing computational effort on recurrently visited states.

The central finding is that horizontal mergers, while reducing static consumer surplus in the short run, create a powerful incentive for entry and investment. Firms enter with negative static profits because the prospect of a lucrative buyout justifies the initial investment. The result is substantially higher long-run innovation and consumer welfare with mergers than without. This finding reverses the standard static antitrust prediction and can only emerge in a dynamic model where firms are forward-looking.

Two related papers merit brief mention. Lomys and Magnolfi (2024) develop structural estimation methods for strategic settings where agents use reinforcement learning rather than playing a fixed equilibrium. They impose an "asymptotic no-regret" condition

[29] We use Π_i for firm i's per-period profit to avoid confusion with policy π.

[30] The Pakes-McGuire algorithm (Pakes and McGuire, 1994) computes Markov perfect equilibria by iterating: (1) given current value functions, compute best-response strategies; (2) given strategies, update value functions via simulation. Convergence is not guaranteed but works well in practice.

as a minimal rationality requirement and derive identification results for payoff parameters. Their contribution is conceptual: they bridge the gap between RL-based behavioral models (where agents learn) and the econometric problem (where the researcher estimates structural parameters from data generated by learning agents). Their estimation method itself is standard econometrics rather than RL. Covarrubias et al. (2022) uses deep RL to study oligopolistic pricing in a New Keynesian framework, representing firms' pricing policies as neural networks $\pi_\phi(a|s)$. The method uncovers multiple equilibria ranging from competitive to collusive pricing.

6.3 Auction Equilibria and Mechanism Design

Computing equilibrium bidding strategies in auctions is a long-standing challenge. Closed-form solutions exist only for narrow families of valuation distributions and auction formats, while numerical methods scale poorly with the number of bidders and items. Recent work has applied deep RL to learn equilibrium strategies and optimal mechanisms.

6.3.1 Brero, Eden, Gerstgrasser, Parkes, and Rheingans-Yoo (2021): RL for Sequential Price Mechanisms

Brero et al. (2021) use RL to design optimal sequential price mechanisms (SPMs), a class of indirect auction mechanisms where agents are approached in sequence and offered menus of items at posted prices. SPMs generalize both serial dictatorship and posted-price mechanisms and essentially characterize all strongly obviously strategyproof (SOSP) mechanisms (Pycia and Troyan, 2023).[31]

The mechanism design problem is formulated as a partially observable Markov decision process (POMDP). The state at round t includes the set of remaining items $\rho_t^{\text{items}} \subseteq [m]$, remaining agents $\rho_t^{\text{agents}} \subseteq [n]$, the partial allocation $\mathbf{x}_t$, and the agents' (unobserved) valuation functions. The action is the choice of which agent to visit next and what prices to offer: $a_t = (i_t, \{p_j^t\}_{j \in \rho_{t-1}^{\text{items}}})$. The observation is the agent's purchase decision, from which the mechanism can update beliefs about valuations. The reward is the objective

[31] A mechanism is strategyproof if truthful reporting is a dominant strategy. It is obviously strategyproof if this dominance is apparent even to boundedly rational agents; strongly obviously strategyproof (SOSP) adds that dominance holds at every information set (Pycia and Troyan, 2023).

function evaluated at the final allocation:

$$r = g(\mathbf{x}_T, \boldsymbol{\tau}_T; \mathbf{v}), \tag{68}$$

where g can be social welfare $\sum_i v_i(x_i)$, revenue $\sum_i \tau_i$, or max-min fairness $\min_i v_i(x_i)$.

A key theoretical result establishes when adaptive mechanisms outperform static ones.

Theorem 4 (Brero et al. (2021), Theorem 1, informal)**.** *For settings with at least two items and correlated agent valuations, there exist valuation distributions such that the optimal SPM with adaptive prices and ordering strictly dominates the optimal mechanism with static (non-adaptive) prices or ordering.*

The policy maps from a sufficient statistic of the observation history to actions. Brero et al. (2021) show that this statistic can be represented compactly in the number of items and agents: it is the set of remaining items and, for each remaining agent, whether agents in each "type class" have purchased or not. They train the mechanism policy using Proximal Policy Optimization (PPO), which handles the discrete action space (agent selection) and continuous action space (price setting) of the POMDP.

Experimental results show that the learned SPMs achieve near-optimal welfare across settings with up to 10 agents and 10 items, significantly outperforming static pricing benchmarks. The improvement is largest when agent valuations are correlated, since adaptive prices allow the mechanism to infer information about remaining agents from earlier purchases.

The limitation is that the POMDP formulation requires knowledge of the prior distribution over valuations, which in practice must be estimated from data. The method also does not scale easily to very large numbers of items due to the combinatorial action space.

6.3.2 Ravindranath, Feng, Wang, Zaheer, Mehta, and Parkes (2024): Differentiable RL for Combinatorial Auctions

Ravindranath et al. (2024) address revenue-maximizing mechanism design for sequential combinatorial auctions with multiple items and strategic bidders. Their innovation is integrating differentiable auction structure into the RL training loop, enabling gradient-based learning where standard RL methods struggle.

The sequential auction proceeds over T rounds. In each round, a subset of items is auctioned. Bidder valuations evolve over rounds (e.g., complementarities between items won in earlier rounds affect valuations for later items). The mechanism's policy determines reserve prices, allocation rules, and payment rules for each round, conditioned on the history of bids and outcomes.

The state at round t is $s_t = (h_t, \mathbf{v}_t)$ where h_t is the public history of past allocations and payments, and $\mathbf{v}_t$ collects bidders' current valuations (partially observed by the mechanism through bids). The mechanism's action is the auction rule for round t: reserve prices, allocation, and payments. The reward is total revenue across all rounds.

The key technical contribution is making the auction clearing and payment rules differentiable. In a standard auction, the allocation is an argmax over bids (non-differentiable), and payments depend discontinuously on the allocation. Ravindranath et al. (2024) replace hard allocations with soft allocations using a softmax relaxation[32] and smooth payment rules, enabling end-to-end gradient computation through the auction. The policy gradient then combines standard RL gradients (for the stochastic components) with first-order gradients through the differentiable auction structure:

$$\nabla_\phi J = \underbrace{\mathbb{E}\left[\sum_t \nabla_\phi \log \pi_\phi(a_t|h_t)\, R_t\right]}_{\text{REINFORCE}} + \underbrace{\mathbb{E}\left[\sum_t \nabla_\phi r_t^{\text{diff}}\right]}_{\text{pathwise gradient}}{}^{33}, \tag{69}$$

where r_t^{diff} is the differentiable revenue from round t and R_t is the return-to-go. This hybrid gradient estimator has lower variance than pure REINFORCE and enables faster convergence.

Experiments demonstrate scaling to environments with up to 50 bidders and 50 items, where the learned mechanisms achieve 15–30% higher revenue than analytical benchmarks (e.g., Myerson optimal auctions applied item-by-item) and conventional deep RL algorithms (PPO, DQN) that do not exploit the differentiable structure. The approach effectively bridges auction theory's optimal mechanisms and practical implementable policies for complex multi-item settings.

[32] The softmax relaxation replaces the hard allocation $argmax_i\, b_i$ with a soft allocation $\exp(b_i/\tau)/\sum_j \exp(b_j/\tau)$, which is differentiable and approaches the hard allocation as $\tau \to 0$.

[33] The pathwise (or reparameterization) gradient computes $\nabla_\phi \mathbb{E}[f(x)]$ by writing $x = g(\phi, \epsilon)$ for noise ϵ independent of ϕ, then differentiating $f(g(\phi, \epsilon))$ directly. This has lower variance than REINFORCE when applicable.

7 Reinforcement Learning in Games

Computing Nash equilibria in extensive-form games with imperfect information is computationally hard. Daskalakis et al. (2009) showed the problem is PPAD-complete, meaning no polynomial-time algorithm is known and none is likely to exist.[34] Despite this worst-case hardness, practical algorithms can exploit game structure. Counterfactual regret minimization (CFR) achieves $O(1/\sqrt{T})$ convergence to ε-Nash equilibrium in two-player zero-sum games (Zinkevich et al., 2008). After T iterations, exploitability decreases as $1/\sqrt{T}$: reaching exploitability 0.01 requires approximately 10,000 iterations.[35] This section applies CFR to the durable goods monopoly problem, benchmarking against the theoretical results of Gul et al. (1986) and Ausubel and Deneckere (1989).

7.1 Counterfactual Regret Minimization

An extensive-form game consists of a game tree with information sets $\mathcal{I}_i$ partitioning player i's decision nodes. An information set groups nodes where the player cannot distinguish between them due to hidden opponent actions or private information. A behavioral strategy $\sigma_i : \mathcal{I}_i \to \Delta(\mathcal{A})$ assigns action probabilities at each information set.

The key concept in CFR is the counterfactual value: how much better would player i have done by taking action a at information set I, assuming she played to reach I but her opponent played according to σ? Formally:

$$v_i^\sigma(I, a) = \sum_{h \in I} \pi_{-i}^\sigma(h) \sum_{z \sqsupseteq ha} \pi^\sigma(ha, z) u_i(z) \tag{70}$$

where $\pi_{-i}^\sigma(h)$ is the probability opponents reach h, and $\pi^\sigma(ha, z)$ is the probability of reaching terminal z from ha. The counterfactual formulation weights by opponent reach $\pi_{-i}^\sigma(h)$ rather than joint reach $\pi^\sigma(h)$. This ensures non-zero updates even at rarely-visited information sets, avoiding the vanishing gradient problem that affects direct policy gradient methods in extensive-form games. Cumulative regret for action a is $R^T(I, a) =$

[34] PPAD (Polynomial Parity Arguments on Directed graphs) is a complexity class for problems where existence is guaranteed by a parity argument. PPAD-completeness means the problem is as hard as any problem in this class. For comparison, checking whether a strategy profile is a Nash equilibrium is easy (polynomial time); finding one is PPAD-complete.

[35] An ε-Nash equilibrium is a strategy profile where no player can improve her expected payoff by more than ε through unilateral deviation. As $\varepsilon \to 0$, this converges to an exact Nash equilibrium.

$\sum_{t=1}^{T}[v^{\sigma^t}(I,a) - v^{\sigma^t}(I)]$. CFR updates via regret matching:[36]

$$\sigma^{T+1}(I,a) = \frac{[R^T(I,a)]^+}{\sum_{a'}[R^T(I,a')]^+}, \quad [x]^+ = \max(x,0). \tag{71}$$

The average strategy $\bar{\sigma}^T$ converges to ε-Nash with $\varepsilon = O(|\mathcal{I}|\sqrt{|\mathcal{A}|}\Delta/\sqrt{T})$ (Zinkevich et al., 2008).[37]

7.2 Neural Extensions

Tabular CFR stores regrets at every information set, infeasible when $|\mathcal{I}| > 10^{14}$. Two neural approaches scale to large games.

7.2.1 Deep CFR

Brown et al. (2019) approximate cumulative regrets with a neural network $V_\theta : \mathcal{I} \times \mathcal{A} \to \mathbb{R}$ trained on sampled (information set, action, regret) tuples:

$$L_V(\theta) = \mathbb{E}_{(I,a,r)\sim\mathcal{M}}\left[(V_\theta(I,a) - r)^2\right]. \tag{72}$$

A separate network Π_ϕ approximates the average strategy via KL-divergence minimization. The current strategy derives from regret matching: $\sigma(I,a) \propto [V_\theta(I,a)]^+$.

7.2.2 Neural Fictitious Self-Play

NFSP (Heinrich and Silver, 2016) combines fictitious play[38] (Brown, 1951) with deep Q-learning.[39] Each player maintains two networks: a best-response network Q_θ trained via DQN, and an average strategy network $\bar{\pi}_\phi$ trained via supervised learning on the

[36] Regret matching is an action selection rule where the probability of choosing action a is proportional to the cumulative regret for not having chosen a in the past (truncated at zero). Actions with high regret receive higher probability; actions with negative regret (having performed worse than average) receive zero probability.

[37] CFR+ (Tammelin, 2014) modifies the update to $R^{T+1}(I,a) = [R^T(I,a) + r^{T+1}(I,a)]^+$ and weights iteration t by t. These modifications improve practical convergence without changing the asymptotic rate.

[38] Fictitious play (Brown, 1951) is a classical learning rule where each player best-responds to the empirical distribution of opponents' past actions. Under certain conditions (e.g., two-player zero-sum games), the time-average strategies converge to Nash equilibrium.

[39] Deep Q-Network (DQN) (Mnih et al., 2015) approximates the Q-function with a neural network, using experience replay and a target network to stabilize training. The target network θ^- in the loss function is a delayed copy of the main network, updated periodically.

best-response actions:

$$L_{\mathrm{RL}}(\theta) = \mathbb{E}\left[\left(r + \gamma \max_{a'} Q_{\theta^-}(I', a') - Q_\theta(I, a)\right)^2\right], \quad L_{\mathrm{SL}}(\phi) = \mathbb{E}\left[-\log \bar{\pi}_\phi(a|I)\right]. \quad (73)$$

During play, agents follow $\bar{\pi}_\phi$ with probability $1 - \eta$ and Q_θ with probability η.

7.2.3 Poker Results

These methods achieved superhuman performance in poker. Deep CFR attained exploitability of 0.036 big blinds per hand[40] in heads-up limit hold'em (Brown et al., 2019). Libratus defeated top human professionals in heads-up no-limit hold'em using nested subgame solving with CFR (Brown and Sandholm, 2018). Pluribus extended to six-player no-limit hold'em (Brown and Sandholm, 2019). The game tree of heads-up no-limit hold'em contains $\sim 10^{161}$ states; these results demonstrate that CFR-based methods scale to economically relevant game sizes.

7.3 The Coase Conjecture

The durable goods monopoly problem has two distinct cases that yield qualitatively different equilibria. In the "gap case," the seller's cost is strictly below all buyer valuations, so trade is guaranteed. In the "no-gap case," cost equals or exceeds the lowest buyer valuation, so some buyers may never trade. The gap case admits a unique stationary equilibrium; the no-gap case admits multiple equilibria with more complex dynamics. We study the gap case, which has a clean theoretical benchmark.

Coase (1972) conjectured that a durable goods monopolist[41] loses market power when buyers are patient: unable to commit to future prices, the seller competes with her future self, eroding rents. Stokey (1981) showed that a monopolist facing rational consumers cannot price discriminate intertemporally; Bulow (1982) proved that in the no-gap case, the monopolist prefers renting to selling. Gul et al. (1986) provided the definitive formalization: with a continuum of buyer types and continuous time, as the inter-offer interval $\Delta t \to 0$, price converges to marginal cost instantaneously and all trade occurs at $t = 0$.

[40] Exploitability measures how far a strategy is from Nash equilibrium: it is the maximum expected gain an adversary could achieve by best-responding. Zero exploitability means the strategy is unexploitable (Nash). In poker, exploitability is measured in milli-big-blinds per hand (mbb/h).

[41] A durable good provides utility over multiple periods (e.g., a car, appliance, or software license) rather than being consumed immediately. Unlike non-durable goods, durable goods create intertemporal competition: the seller at time t competes with her own future self at $t + 1$.

7.3.1 Model

A seller with zero cost faces a buyer with private valuation $v \in \{v_L, v_H\}$, where $\Pr(v = v_H) = \pi$. In each period $t = 1, 2, \ldots$, the seller posts price p_t; the buyer accepts or rejects. Payoffs upon acceptance at t: seller receives $\delta^{t-1} p_t$, buyer receives $\delta^{t-1}(v - p_t)$, where $\delta \in (0, 1)$ is the common discount factor. The game ends upon acceptance or after T periods.

This is the gap case: seller cost (0) is strictly below the lowest buyer valuation (v_L), guaranteeing trade in equilibrium.

7.3.2 Theoretical Benchmark

The screening price $P^*(\delta)$ makes the high-type buyer indifferent between accepting now and waiting:

$$v_H - P^* = \delta(v_H - v_L) \implies P^*(\delta) = v_H - \delta(v_H - v_L). \tag{74}$$

The seller's optimal strategy depends on π. Let $\Pi_{\text{screen}} = \pi P^* + (1-\pi)\delta v_L$ and $\Pi_{\text{pool}} = v_L$. The seller screens if $\Pi_{\text{screen}} > \Pi_{\text{pool}}$, yielding threshold

$$\pi^* = \frac{v_L(1-\delta)}{P^* - \delta v_L} = \frac{1}{2} \quad \text{when } v_H = 2v_L. \tag{75}$$

For $\pi < \pi^*$: pooling equilibrium (offer v_L immediately). For $\pi > \pi^*$: screening equilibrium (offer P^*, then v_L if rejected).[42]

The Coase conjecture manifests as $\delta \to 1$: the screening price $P^*(\delta) \to v_L$, so the seller cannot extract surplus from high types.

7.3.3 Computational Results

We model the bargaining game as an extensive-form game and apply CFR. Parameters: $v_L = 100$, $v_H = 200$, $T = 2$ periods. The seller's information set is the rejection history; the buyer's information set includes her private type.

Table 14 reports results from varying π at fixed $\delta = 0.5$. The algorithm recovers the phase transition at $\pi^* = 0.5$: for $\pi < 0.5$, the seller pools with probability approaching

[42] In a screening equilibrium, the seller uses price to separate buyer types: high-value buyers accept immediately at a high price, while low-value buyers reject and receive a lower offer later. In a pooling equilibrium, the seller offers a single price that all types accept, sacrificing price discrimination for immediate sale.

Table 14: CFR Equilibrium vs. Theory: π-sweep at $\delta = 0.5$

π	P^*	P(Screen)	Theory	NashConv	Eq. Type	Status
0.10	150	0.000	0.0	5.0258	Pooling	✓
0.15	150	0.000	0.0	7.5249	Pooling	✓
0.20	150	0.000	0.0	10.0239	Pooling	✓
0.25	150	0.000	0.0	12.5229	Pooling	✓
0.30	150	0.000	0.0	15.0223	Pooling	✓
0.35	150	0.000	0.0	17.5216	Pooling	✓
0.40	150	0.000	0.0	20.0213	Pooling	✓
0.45	150	0.000	0.0	22.5213	Pooling	✓
0.50	150	0.001	0.5	25.0221	Indifferent	✓
0.55	150	0.002	1.0	27.5256	Screening	✓
0.60	150	0.506	1.0	30.0066	Screening	✓
0.65	150	0.999	1.0	26.2553	Screening	✓
0.70	150	1.000	1.0	22.5027	Screening	✓
0.75	150	1.000	1.0	18.7531	Screening	✓
0.80	150	1.000	1.0	15.0067	Screening	✓
0.85	150	1.000	1.0	11.2576	Screening	✓
0.90	150	1.000	1.0	7.5089	Screening	✓

Notes: P(Screen) is the probability the seller offers $P^* = 150$ in period 1. Theory column gives the predicted probability (0 for pooling, 1 for screening). NashConv is the sum of exploitabilities. 5,000 CFR iterations per configuration.

1; for $\pi > 0.5$, the seller screens with probability approaching 1. The transition is sharp.

A δ-sweep at $\pi = 0.7$ confirms the screening price formula: $P^*(\delta) = 200 - 100\delta$ with zero error across $\delta \in [0.1, 0.9]$. As δ increases, screening becomes unprofitable (Coase conjecture): at $\delta = 0.75$, the seller switches to pooling.

7.3.4 Validation

Four stress tests verify the results.[43] The 3-period result merits explanation. At $P_1 = 136$, the high buyer is exactly indifferent: surplus $= v_H - 136 = 64 = \delta(v_H - 120)$. If the buyer randomizes, the seller prefers $P_1 = 120$ (revenue 108 vs. 86.4 if buyer rejects 136). CFR found the equilibrium robust to tie-breaking.

7.4 Discussion

CFR computed equilibria matching the theoretical predictions of Gul et al. (1986) without encoding economic structure. The algorithm discovered that: (1) the seller screens when

[43] (1) Awkward primes $(v_L, v_H) = (37, 83)$: converged to $P^* = 55$ (theory: 55.4). (2) Information leak: single seller information set at root. (3) Grid shift with 150 removed: used 145 (99.4%). (4) 3-period game: found $P_1 = 120$ vs. theory 136.

π is high, pools when π is low, with threshold at $\pi^* = 0.5$; (2) the screening price satisfies the high-buyer indifference condition; (3) as $\delta \to 1$, screening collapses (Coase conjecture). These properties emerged from regret minimization alone.

The poker-bargaining correspondence is exact: buyer valuation maps to hole cards, price acceptance to calling a bet, rejection to folding. Both involve signaling private information through sequential actions. The success of poker algorithms on this benchmark suggests applicability to other bargaining and mechanism design problems with analytically intractable equilibria.

8 Multi-Armed Bandits in Economics

Multi-armed bandits represent the simplest form of reinforcement learning: sequential decision-making without state. The agent repeatedly chooses among K actions, observes a stochastic reward, and updates its beliefs. This stateless structure makes bandits uniquely suited for real-world deployment where the cost of suboptimal decisions is immediate and irreversible. The central challenge is the exploration-exploitation tradeoff: the agent must balance gathering information about uncertain arms against exploiting arms that appear best. Two algorithmic paradigms dominate. The Gittins index, derived from dynamic programming under discounting, provides the Bayes-optimal solution for independent arms but requires solving a stopping problem for each arm at each period.[44] Upper Confidence Bound (UCB) algorithms offer a frequentist alternative with finite-time regret guarantees and minimal computational overhead. This chapter explores how economic structure improves upon generic bandit algorithms. When the reward function satisfies monotonicity, unimodality, or revealed preference constraints from consumer theory, algorithms can eliminate suboptimal arms faster and achieve logarithmic rather than square-root regret scaling.

A K-armed bandit is an MDP with a single state. The action set is $\mathcal{A} = \{1, \ldots, K\}$,

[44] Gittins (1979) showed that the optimal policy for the discounted infinite-horizon bandit has an index form: compute a scalar index for each arm based on its posterior, then pull the arm with highest index. The index equals the fair subsidy that makes the agent indifferent between pulling the arm and receiving a guaranteed payment.

and pulling arm a yields reward $r_t \sim \nu_a$ with mean μ_a.[45] The standard metric is regret:

$$R(T) = T \cdot \max_i \mu_i - \mathbb{E}\left[\sum_{t=1}^{T} r_t\right].$$

Regret measures the profit lost due to not knowing the optimal arm. Standard algorithms achieve $O(\sqrt{KT})$ regret. Economic structure can improve this to $O(\log T)$.

8.1 Dynamic Pricing with Demand Learning

A seller faces sequential customers with unknown demand. Setting price $p \in \mathcal{P} = \{p_1, \ldots, p_K\}$ yields sale probability $\theta(p)$ and revenue $R_t = p \cdot \mathbb{1}\{\text{sale}\}$. Microeconomic theory provides structure: $\theta(p)$ decreases monotonically in p.[46] Under the monotone hazard rate condition,[47] expected revenue $\mu(p) = p \cdot \theta(p)$ is unimodal.

Misra et al. (2019) exploit unimodality with a modified UCB algorithm[48] that maintains a leader arm and explores only the leader and its neighbors:

$$I_p(t) = \hat{\mu}_p(t) + \sqrt{\frac{2 \ln t}{N_p(t)}} \tag{76}$$

where $\hat{\mu}_p(t)$ is average revenue and $N_p(t)$ the number of trials at price p. When a neighbor's index exceeds the leader's, the algorithm shifts focus. This achieves $O(\log T)$ profit loss versus $O(\sqrt{KT})$ for standard UCB.[49] We implement this algorithm in Section 8.4.

For multi-product settings, Mueller et al. (2019) impose low-rank structure on the price-sensitivity matrix $\mathbf{B}$ in the demand system $\mathbf{q}_t = \mathbf{c}_t - \mathbf{B}\mathbf{p}_t + \boldsymbol{\varepsilon}_t$, achieving profit loss $O(T^{3/4}\sqrt{d})$ scaling with latent dimension $d \ll N$.[50]

[45] Formally, $|\mathcal{S}| = 1$, so there is no state transition or discounting. Unlike full RL which runs in simulation, bandits deploy in real-time where sample efficiency is critical.

[46] Consumers purchase when valuation exceeds price. If valuations follow distribution F, then $\theta(p) = 1 - F(p)$.

[47] The hazard rate of distribution F is $h(v) = f(v)/(1 - F(v))$. The monotone hazard rate (MHR) condition requires $h(v)$ to be non-decreasing. Many common distributions (uniform, exponential, normal) satisfy MHR. Under MHR, revenue $p(1 - F(p))$ is strictly quasi-concave, ensuring a unique optimal price.

[48] Upper Confidence Bound (UCB) algorithms balance exploration and exploitation by selecting the arm with highest upper confidence bound $\hat{\mu}_a + c\sqrt{\ln t / N_a}$, where $\hat{\mu}_a$ is the empirical mean, N_a the number of pulls, and c controls the exploration bonus. Arms with high uncertainty or high estimated reward receive priority.

[49] Field experiments at an online retailer showed 43% higher profits versus static pricing.

[50] Xu et al. (2021) achieve $O(d \log T)$ for contextual pricing with linear valuations $v_t = \boldsymbol{\theta}^\top \mathbf{x}_t$. Goyal et al. (2022) handle MNL choice models (multinomial logit extends the binary logit to $K > 2$ alternatives; in pricing, MNL models consumer choice among differentiated products where utility depends on product attributes and price), achieving $O(d\sqrt{T} \log T)$ under adversarial contexts.

8.2 Bid Optimization in Auctions

In a second-price auction[51] with reserve r, revenue is $b_{(2)}\mathbb{1}\{b_{(2)} > r\} + r\mathbb{1}\{b_{(1)} \geq r > b_{(2)}\}$. Myerson's theory establishes that expected revenue is unimodal in r under regularity conditions.[52] Cesa-Bianchi et al. (2015) leverage this with a unimodal bandit algorithm, achieving $O(\log T)$ profit loss by eliminating provably suboptimal regions.[53]

8.3 Strategic Interactions

Guo et al. (2023) study multi-firm pricing where consumers follow a logit model[54] with reference price effects:[55]

$$U_t(i) = \alpha_i - \beta p_{i,t} + \gamma(\rho_t - p_{i,t}) + \epsilon_{i,t}, \quad \rho_{t+1} = (1-\delta)\rho_t + \frac{\delta}{n}\sum_i p_{i,t} \tag{77}$$

Online projected gradient ascent achieves $O(1/t)$ convergence to Nash equilibrium, faster than model-free approaches.

8.4 Simulation Study: Dynamic Pricing with Partial Identification

We replicate the simulation of Misra et al. (2019) to evaluate UCB-PI against standard bandit algorithms. The environment has $K = 100$ prices on a grid from \$0.01 to \$1.00, $S = 1{,}000$ consumer segments with equal weights, and within-segment heterogeneity $\delta = 0.1$. Each segment s has midpoint valuation $v_s \sim \text{Uniform}(0.1, 0.9)$, and consumer i in segment s has valuation $v_i \sim \text{Uniform}(v_s - \delta, v_s + \delta)$. A consumer purchases if and only if $v_i \geq p$ (WARP). The optimal price is $p^* = 0.45$ with expected profit $\mu^* = 0.248$ per customer. We run $T = 200{,}000$ rounds across 10 seeds.

We compare five algorithms spanning the model-free to model-based spectrum. Ta-

[51] In a second-price (Vickrey) auction, bidders submit sealed bids and the highest bidder wins, paying the second-highest bid. This mechanism is incentive-compatible: bidding one's true valuation is a weakly dominant strategy regardless of others' behavior.

[52] Myerson (1981) shows the optimal reserve price satisfies $r^* = \psi^{-1}(0)$ where $\psi(v) = v - (1 - F(v))/f(v)$ is the virtual valuation. Regularity requires $\psi(v)$ to be strictly increasing, which holds under the MHR condition.

[53] Akcay et al. (2022) address censored feedback where losing bids are unobserved, using auction rules to convert partial observations into side information.

[54] The logit (multinomial logit) model specifies choice probabilities as $\Pr(\text{choose } i) = e^{U_i}/\sum_j e^{U_j}$, where U_i is the utility of option i. It derives from assuming i.i.d. Type-I extreme value (Gumbel) errors $\epsilon_{i,t}$. The model implies the independence of irrelevant alternatives (IIA) property.

[55] Reference price effects capture the behavioral finding that consumers evaluate prices relative to an anchor. The reference price ρ_t represents the expected price level; a current price below ρ_t feels like a gain, boosting purchase probability beyond what absolute price alone would predict.

ble 15 summarizes their theoretical properties. UCB1 is the standard upper confidence bound algorithm with no structural assumptions. Thompson Sampling maintains Beta posteriors over arm means and samples to select actions. Learn-Then-Exploit (LTE) uses a pure exploration phase followed by greedy exploitation. UCB-PI incorporates the partial identification mechanism of Misra et al. (2019), eliminating dominated prices using WARP bounds. UCB-PI-tuned uses a more conservative dominance threshold (2δ vs. δ) to reduce false eliminations.

Table 15: Algorithm classification by structure exploitation. Regret rates are theoretical under standard assumptions; K is the number of arms, T the horizon.

Algorithm	Uses WARP	Uses Unimodality	Regret Rate
UCB1	No	No	$O(\sqrt{KT})$
Thompson	No	No	$O(\sqrt{KT})$
LTE (5%)	No	No	$O(T^{2/3})$
UCB-PI	Yes	Implicit	$O(\log T)$
UCB-PI-tuned	Yes	Implicit	$O(\log T)$

Table 16 reports cumulative profits and ex-post profit ratios. UCB-PI-tuned achieves the highest mean profit ratio at 99.4%, followed by Thompson Sampling at 97.9%. The partial identification mechanism eliminates dominated prices: by $T = 200{,}000$, UCB-PI has eliminated 37 of 100 prices on average, while UCB-PI-tuned eliminates 15.

Table 16: Dynamic pricing with partial identification ($K = 100$ prices, $S = 1{,}000$ segments, $T = 200{,}000$ customers, 10 seeds). Frac. Opt is the fraction of rounds at p^*. Dominated is mean number of prices eliminated by partial identification.

Algorithm	Ex Post %	Range	Profit	Frac. Opt	Dominated
Oracle	100.0%	[100-100%]	50701	0.200	–
UCB-PI-tuned	99.4%	[99-100%]	50387 ± 658	0.104	15
UCB-PI	92.4%	[91-93%]	46831 ± 724	0.048	37
UCB1	86.8%	[86-87%]	44024 ± 597	0.028	–
LTE (5%)	95.6%	[92-98%]	48510 ± 1123	0.000	–
Thompson	97.9%	[98-98%]	49634 ± 657	0.094	–

Figure 10 shows cumulative profit trajectories. All learning algorithms converge toward oracle performance, with UCB-PI-tuned tracking the oracle most closely throughout.

Figure 11 validates the theoretical regret rates. When regret is divided by $\log(t)$, algorithms with $O(\log T)$ regret should stabilize while $O(\sqrt{T})$ algorithms diverge. UCB-

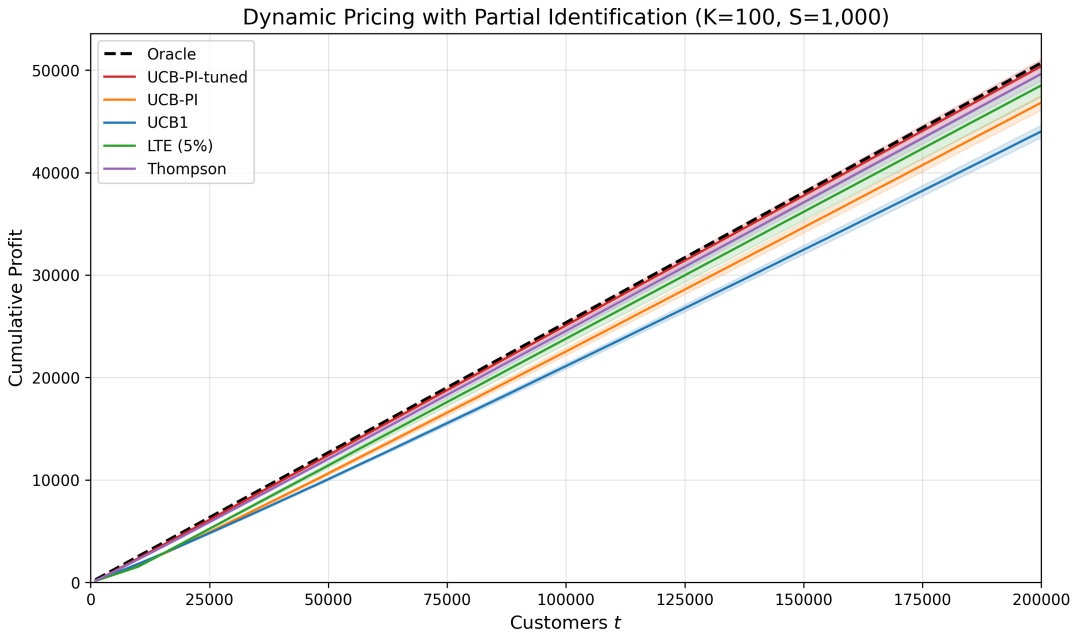

Figure 10: Cumulative profit for dynamic pricing with partial identification. Oracle (dashed black) knows demand perfectly. Shaded regions show ± 2 standard errors across 10 seeds.

PI-tuned stabilizes near 26, consistent with logarithmic regret. UCB1 and Thompson grow without bound, consistent with $O(\sqrt{T})$. UCB-PI shows intermediate behavior: the dominance mechanism reduces regret but overly aggressive elimination (37 prices dominated vs. 15 for tuned) causes some suboptimality.

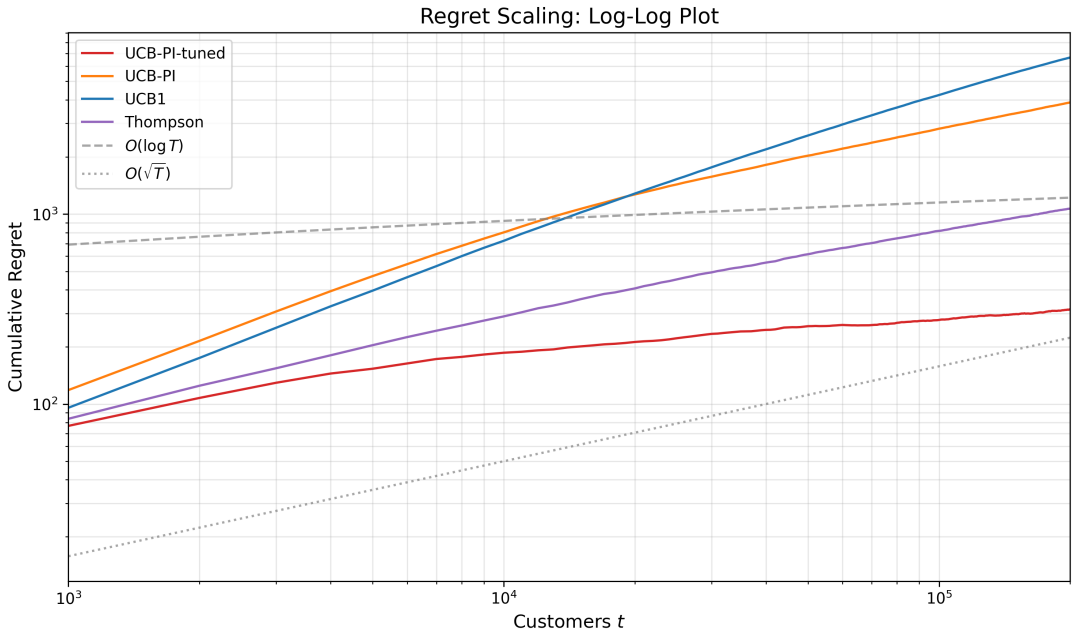

Figure 11: Regret scaling validation. Profit loss divided by $\log(t)$ should stabilize for $O(\log T)$ algorithms and grow for $O(\sqrt{T})$ algorithms.

Figure 12 shows the distribution of price selections across the horizon. UCB-PI-tuned concentrates selections around the optimal price $p^* = 0.45$, with the distribution tightening as more prices are eliminated. UCB1 spreads selections more broadly, reflecting continued exploration across all arms.

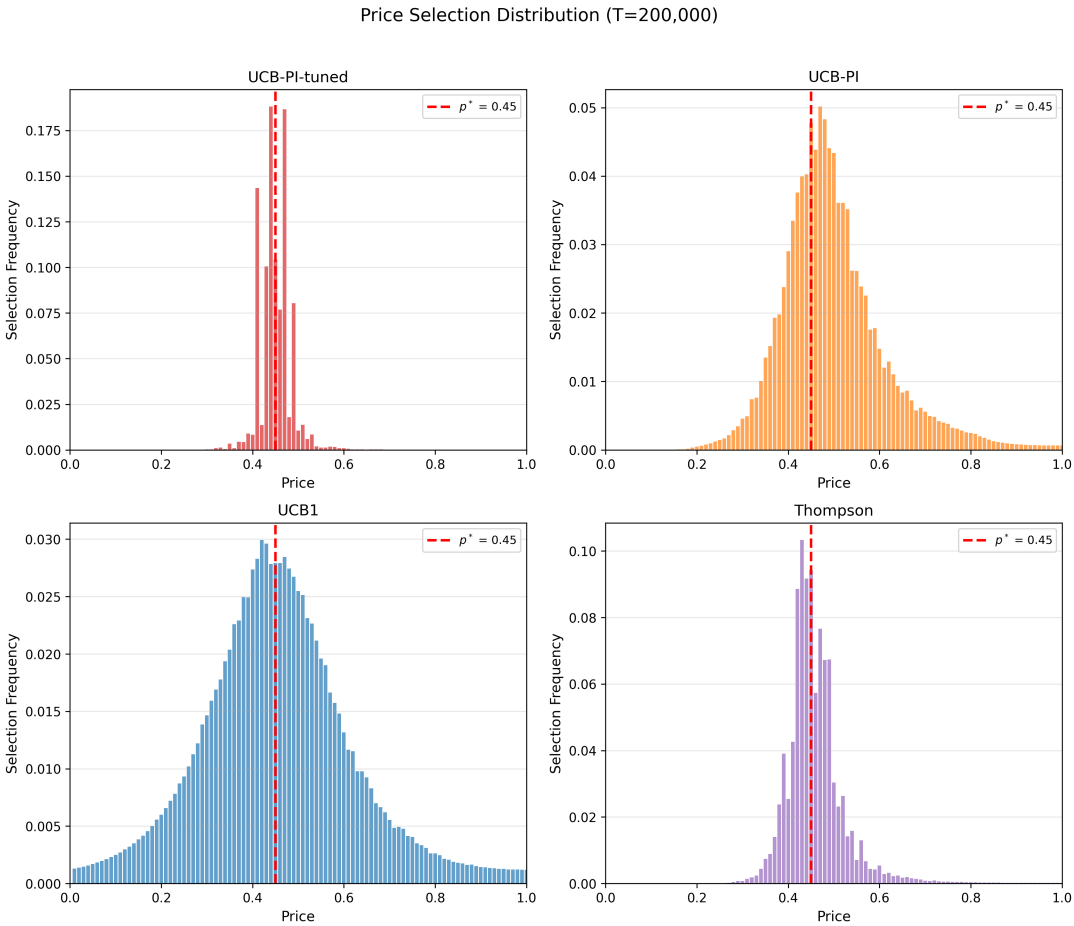

Figure 12: Price selection distribution by algorithm. Vertical dashed line marks optimal price $p^* = 0.45$.

Figure 13 shows the convergence trajectory using a moving average of the selected price. UCB-PI-tuned converges to near p^* within the first 50,000 rounds and remains stable. The standard UCB-PI converges more slowly due to aggressive dominance that occasionally eliminates near-optimal prices early.

9 Reinforcement Learning from Human Feedback

The structural estimation and IRL literatures recover an agent's underlying reward function from observations of behavior. Reinforcement Learning from Human Feedback

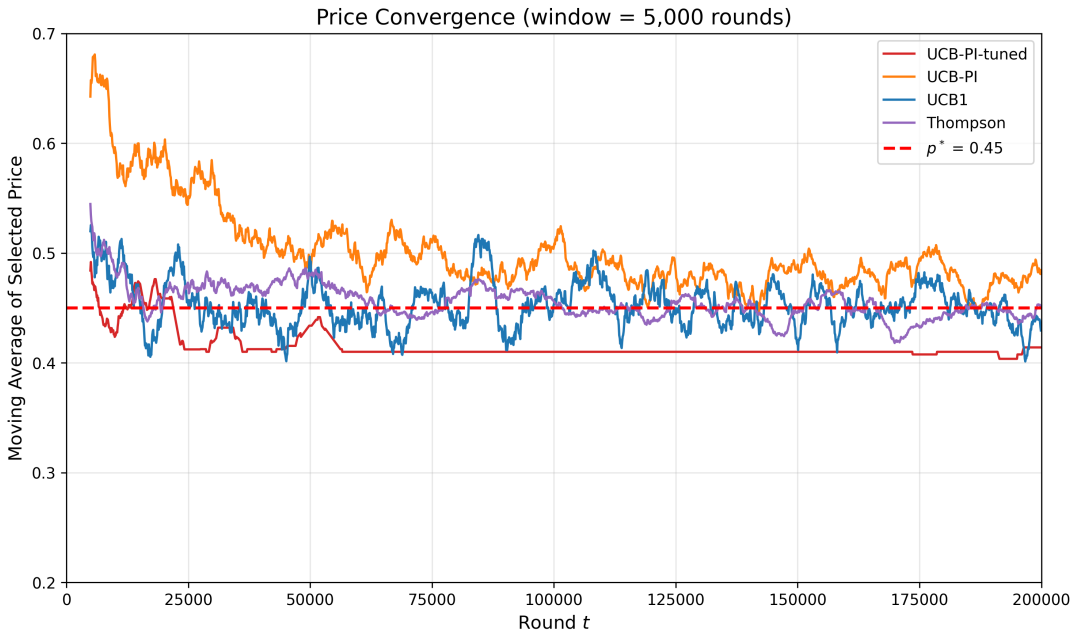

Figure 13: Convergence trajectory (moving average of selected price, window = 1000). Horizontal dashed line marks optimal price $p^* = 0.45$.

(RLHF) addresses a different setting: instead of inverting an observed policy to find the reward function that rationalizes it, RLHF learns a reward function by eliciting preferences from a human evaluator over pairs of machine-generated outputs. Christiano et al. (2017) demonstrated that this approach could train agents without an explicit reward function. RLHF has since become the predominant method for aligning large language models.

9.1 Learning Rewards from Preferences

The canonical RLHF framework is built on a formal model of human preference. The data we observe consists of tuples (s, y_w, y_l), where s is a context, and y_w and y_l are two outputs, with y_w being the "winner" preferred by a human. If we assume preferences follow a latent utility model, the Bradley-Terry model gives the probability that y_w is preferred: $P(y_w \succ y_l|s) = \sigma(r_\theta(s, y_w) - r_\theta(s, y_l))$, where $r_\theta : \mathcal{S} \times \mathcal{Y} \to \mathbb{R}$ is a learned reward model parameterized by θ, trained to approximate human preferences (not the ground-truth reward, which is unobserved), and $\sigma(\cdot)$ is the logistic function. This formulation is a direct application of discrete choice theory and is equivalent to a binary logit model, a special case of the multinomial logit framework (McFadden, 1974) that underpins DDC estimation. The reward model parameters θ are estimated by minimizing the negative

log-likelihood of the observed human choices:

$$\mathcal{L}(\theta) = -\mathbb{E}_{(s,y_w,y_l)\sim\mathcal{D}}\left[\log\sigma\left(r_\theta(s,y_w) - r_\theta(s,y_l)\right)\right], \tag{78}$$

where $\sigma(\cdot)$ is the logistic sigmoid function.

9.2 The RLHF Pipeline and Direct Optimization

We then use the learned r_θ as a proxy objective for policy optimization. The canonical approach, solidified in a series of works applying RLHF to language models (Ziegler et al., 2019; Stiennon et al., 2020; Ouyang et al., 2022), is a three-stage process. First, a base pretrained model is initialized via supervised fine-tuning (SFT) on a small dataset of high-quality demonstrations, yielding an initial policy π^{SFT}. This grounds the model in the desired style and format. The second step is the reward model training as described, using preference data generated from this π^{SFT}.

In the third and final step, we optimize the policy using reinforcement learning. The π^{SFT} policy, now denoted π_ϕ, is fine-tuned to maximize the expected reward signal from the frozen reward model r_θ. A critical component of this optimization is the inclusion of a regularization term that penalizes the Kullback-Leibler (KL) divergence between the learned policy π_ϕ and the initial reference policy π^{SFT}. The objective function for this final stage is:

$$J(\phi) = \mathbb{E}_{s\sim\mathcal{D},y\sim\pi_\phi(\cdot|s)}[r_\theta(s,y)] - \lambda_{KL}\mathbb{E}_{s\sim\mathcal{D}}[D_{KL}(\pi_\phi(\cdot|s)\,||\,\pi^{SFT}(\cdot|s))], \tag{79}$$

where D_{KL} is the Kullback-Leibler divergence and λ_{KL} is a hyperparameter controlling the penalty strength.[56] This KL-divergence term is essential: it prevents the policy π_ϕ from deviating too far from the distribution of text on which the reward model is reliable, thus mitigating "reward hacking" where the policy exploits inaccuracies in r_θ to achieve a high score with degenerate behavior.

While empirically successful, the KL-regularized objective in Equation (79) can seem ad-hoc. However, a more principled interpretation frames the entire process as a form of Bayesian inference (Korbak et al., 2022). In this view, the reference policy π^{SFT} acts as a prior distribution over plausible responses. The reward model r_θ provides evidence,

[56] We use λ_{KL} for the KL penalty weight, reserving β for model parameters and γ for discount factors.

specifying which responses are more desirable. The goal of alignment is to find the posterior distribution π^* that optimally combines the prior with this evidence. This ideal posterior policy takes the form:

$$\pi^*(y|s) \propto \pi^{SFT}(y|s) \exp\left(\frac{r_\theta(s,y)}{\lambda_{KL}}\right). \tag{80}$$

This formulation directly parallels Bayes' rule, where the posterior is proportional to the prior times the likelihood. The reward function, scaled by λ_{KL}, defines the log-likelihood of the preference evidence. From this perspective, the KL-regularized RL objective $J(\phi)$ is no longer arbitrary. It is mathematically equivalent to the Evidence Lower Bound (ELBO) for this Bayesian inference problem. Consequently, maximizing the RLHF objective via PPO is a form of variational inference, where the aim is to find the policy π_ϕ that minimizes the KL divergence to the ideal posterior π^*. This insight reframes RLHF from a pure reward-maximization task into a principled probabilistic inference problem, providing a solid theoretical justification for the KL penalty as a fundamental component rather than a mere regularizer.

Despite this elegance, the three-stage RLHF pipeline is complex to implement, requiring training multiple large models and a computationally expensive reinforcement learning loop that involves sampling from the policy. Building on the analytical relationship between the optimal policy and the reward function, Direct Preference Optimization (DPO) was introduced by Rafailov et al. (2023) as a simpler, RL-free method. DPO bypasses the explicit reward modeling step by reparameterizing the reward function directly in terms of the optimal policy π^* and the reference policy π^{SFT}:

$$r(s,y) = \lambda_{KL} \log\left(\frac{\pi^*(y|s)}{\pi^{SFT}(y|s)}\right) + \lambda_{KL} \log Z(s). \tag{81}$$

When this analytical expression for the reward is substituted into the Bradley-Terry preference loss from Equation (78), the unknown partition function $Z(s)$ cancels out. This yields a loss function that depends only on the policy π_ϕ being optimized and the fixed reference policy π^{SFT}. The DPO objective is thus a simple binary cross-entropy

loss over policy likelihoods:

$$\mathcal{L}_{DPO}(\phi; \pi^{SFT}) = -\mathbb{E}_{(s,y_w,y_l)\sim\mathcal{D}}\left[\log\sigma\left(\lambda_{KL}\log\frac{\pi_\phi(y_w|s)}{\pi^{SFT}(y_w|s)} - \lambda_{KL}\log\frac{\pi_\phi(y_l|s)}{\pi^{SFT}(y_l|s)}\right)\right]. \tag{82}$$

This objective can be optimized directly on the policy parameters ϕ using standard supervised learning techniques with a static preference dataset. The gradient of this loss intuitively increases the likelihood of preferred responses y_w while decreasing the likelihood of dispreferred responses y_l, relative to the reference policy. By recasting the constrained optimization problem as a simple likelihood maximization, DPO offers a more stable and computationally efficient alternative to the multi-stage RLHF pipeline, eliminating the need for an explicitly trained reward model and the complexities of RL-based policy optimization.

9.3 Recent Developments

A subsequent innovation leverages DPO to create an iterative self-improvement loop, reducing reliance on static, human-annotated data (Yuan et al., 2024). In this paradigm, dubbed Self-Rewarding Language Models, a single language model acts as both the instruction-following agent and its own reward model. The process begins with a base model fine-tuned to have both instruction-following and evaluation capabilities ("LLM-as-a-Judge"). In each subsequent iteration, the current model generates a new preference dataset for itself by producing multiple responses to a set of prompts and then evaluating its own outputs to assign scores. These scores are used to construct preference pairs (y_w, y_l), which form an AI-generated feedback dataset. The model is then fine-tuned on this new data using the DPO loss. This iterative process creates a feedback loop: enhancements in instruction-following ability lead to better reward modeling, which in turn fuels the next round of policy optimization.

While RLHF and its successors have proven effective at aligning model behavior with human preferences, several open issues remain. The framework does not aim to solve the identification problem in the strict econometric sense; the learned reward model r_θ (whether explicit or implicit in DPO) is a proxy for preference, not necessarily the uniquely identified "true" utility function required for welfare analysis.[57] Furthermore,

[57]The identification problem in preference learning mirrors that in discrete choice: the reward function is identified only up to an additive constant and requires a location normalization.

the notion of "human feedback" is a significant simplification, as the preferences being optimized are those of a small, non-representative group of paid labelers, raising important questions about whose values are being embedded in these systems. Finally, the fine-tuning process can lead to degraded performance on standard academic benchmarks, a phenomenon called the "alignment tax" (Ouyang et al., 2022).[58] Mitigating these challenges while scaling alignment beyond the limits of human data collection remains a key frontier for the field.

9.4 Simulation Study: Bradley-Terry Preference Learning

We validate the Bradley-Terry preference model from Section 7.1 in a controlled setting where ground truth utilities $w^* = [0.1, 0.3, 0.5, 0.7, 0.9]$ over $K = 5$ options are known. Preferences follow the Bradley-Terry model $\mathbb{P}(a_1 \succ a_2) = \sigma(w^*[a_1] - w^*[a_2])$ with $w[0] = 0$ for identification, mirroring the normalization required in discrete choice estimation (McFadden, 1974). We vary sample size $N \in \{50, 100, 200, 500, 1000\}$ and label noise $\varepsilon \in \{0, 0.05, 0.1, 0.2, 0.3\}$, comparing explicit reward modeling, DPO, and MLE across 20 seeds per configuration.

Table 17 reports the results. MSE scales as $O(1/N)$, consistent with MLE asymptotics for discrete choice models; empirical variance matches the Fisher Information bound to within 10% at $N = 500$. DPO and explicit RM produce identical policies (max difference $< 10^{-6}$), confirming their mathematical equivalence under uniform reference. Weight recovery degrades gracefully under label noise up to $\varepsilon \approx 0.2$, beyond which rank correlation falls below 0.8. Figure 14 shows the convergence of MSE with sample size.

10 Conclusion

This survey has examined reinforcement learning through the lens of economics. The central theoretical result is the unified view of planning and learning: RL extends dynamic programming to large state spaces and model-free settings where transition probabilities and reward functions are unknown. Value iteration, policy iteration, Q-learning, and policy gradients all seek fixed points of operators derived from the Bellman equation.

[58] The alignment tax refers to the tradeoff between helpfulness (as measured by benchmark performance) and harmlessness (as measured by alignment evaluations). Models optimized for safety may refuse to answer questions they could otherwise answer correctly.

Table 17: Preference learning results (20 seeds, $K = 5$ options, $w^* = [0.1, 0.3, 0.5, 0.7, 0.9]$)

Experiment	Configuration	MSE	Rank Corr.	Top-1	Regret
Sample Size	$N = 50$	0.076 ± 0.013	0.54 ± 0.09	35%	—
	$N = 100$	0.051 ± 0.009	0.67 ± 0.06	55%	—
	$N = 200$	0.023 ± 0.003	0.85 ± 0.03	75%	—
	$N = 500$	0.011 ± 0.001	0.93 ± 0.02	75%	—
	$N = 1000$	0.005 ± 0.001	0.99 ± 0.01	100%	—
Method ($N = 500$)	Explicit RM	0.011 ± 0.001	0.93 ± 0.02	75%	0.324 ± 0.003
	DPO	0.011 ± 0.001	0.93 ± 0.02	75%	0.324 ± 0.003
	MLE	0.011 ± 0.001	0.93 ± 0.02	75%	0.324 ± 0.003
Noise ($N = 500$)	$\varepsilon = 0.00$	0.011 ± 0.001	0.93 ± 0.02	75%	0.324 ± 0.003
	$\varepsilon = 0.05$	0.014 ± 0.002	0.93 ± 0.02	70%	0.332 ± 0.003
	$\varepsilon = 0.10$	0.018 ± 0.002	0.92 ± 0.02	65%	0.340 ± 0.003
	$\varepsilon = 0.20$	0.030 ± 0.004	0.80 ± 0.03	50%	0.357 ± 0.003
	$\varepsilon = 0.30$	0.055 ± 0.008	0.68 ± 0.05	50%	0.372 ± 0.003

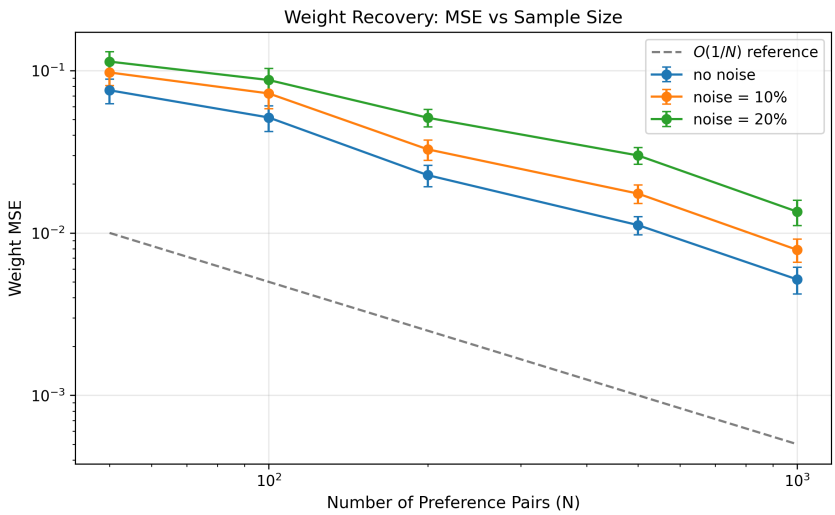

Figure 14: Weight recovery MSE versus sample size on log-log axes. The $O(1/N)$ reference line (dashed) confirms asymptotic efficiency. Higher noise levels shift curves upward but preserve the scaling rate.

The difference is whether that operator is applied with known transitions (planning) or sampled ones (learning).

Key Achievements by Chapter

Chapter 2 established the operator framework connecting RL algorithms to classical DP methods. Policy Iteration achieves superlinear convergence by solving a linearized problem at each step; Value Iteration and Q-learning achieve only linear convergence at rate γ. This explains the $(1-\gamma)^{-1}$ sample complexity gap between model-based and model-free methods.

Chapter 3 demonstrated that RL matches exact DP on tractable benchmarks and outperforms practitioner heuristics where DP is infeasible. Across nine benchmarks spanning dispatch, pricing, execution, inventory, and control, RL policies learned state-dependent strategies that simple rules cannot express.

Chapter 4 showed how RL algorithms can be embedded in structural estimation pipelines. TD learning estimates the recursive value terms in CCP-based DDC estimation without requiring transition density estimation. Policy gradient methods solve the inner-loop DP problem in simulation-based estimators.

Chapter 5 applied counterfactual regret minimization to the durable goods monopoly problem, recovering the theoretical predictions of Gul-Sonnenschein-Wilson without encoding economic structure. The algorithm discovered screening versus pooling equilibria as emergent behavior from regret minimization.

Chapter 6 showed how economic structure (unimodal revenue, monotone demand) yields tighter regret bounds than generic bandit algorithms. Exploiting structure reduces regret from $O(\sqrt{KT})$ to $O(\log T)$.

Chapter 7 connected RLHF to discrete choice econometrics. The Bradley-Terry preference model underlying reward learning is a binary logit. DPO and explicit reward modeling produce equivalent policies under appropriate conditions.

Open Theoretical Problems

Several theoretical questions remain unresolved. First, the sample complexity of RL with function approximation is not fully characterized. Tabular bounds show model-free methods require $O((1-\gamma)^{-4})$ samples versus $O((1-\gamma)^{-3})$ for model-based, but these bounds

may not extend to neural network approximators. Second, convergence guarantees for actor-critic methods with nonlinear function approximation remain incomplete. Third, the relationship between exploration and regret in economic environments with strategic agents is poorly understood.

Methodological Challenges

Four methodological challenges limit current applications. First, reward misspecification: RL optimizes the stated reward function, which may diverge from the economic objective. Second, sample efficiency: in many economic applications, each sample requires costly experimentation or simulation time. Third, safety constraints: production systems require guarantees that RL will not take catastrophic actions during learning. Fourth, interpretability: RL policies are typically black boxes, limiting adoption in regulated industries.

Directions for Integration

The most productive path forward combines economic theory with machine learning methods. Economic theory provides structure that constrains the hypothesis space: demand monotonicity, equilibrium conditions, and behavioral regularities all reduce the effective complexity of learning problems. Machine learning provides computational tools to solve models that were previously intractable: high-dimensional state spaces, continuous actions, and complex strategic interactions.

Specific opportunities include: incorporating equilibrium constraints into multi-agent RL to ensure learned policies constitute Nash equilibria; using revealed preference theory to provide sample-efficient reward learning; applying mechanism design principles to shape the reward structure of RL systems; and developing RL methods that respect budget constraints, fairness requirements, and other economic restrictions.

This bidirectional flow of ideas, already visible in the work surveyed here, will continue to shape both fields.

References

S. Adusumilli, M. Eckardt, and G. Tate. Estimation of dynamic discrete choice models with differentiable temporal-difference learning. *arXiv preprint arXiv:2209.15174*, 2022.

Alekh Agarwal, Sham M. Kakade, Jason D. Lee, and Gaurav Mahajan. On the theory of policy gradient methods: Optimality, approximation, and distribution shift. *Journal of Machine Learning Research*, 22(98), 2021.

Alp Akcay, Onur Atan, Muhammed O. Sayin, and Atilla Eryilmaz. Online learning algorithms for auction reserve price optimization with censored feedback. *Operations Research*, 2022.

Robert Almgren and Neil Chriss. Optimal execution of portfolio transactions. *Journal of Risk*, 3(2):5–40, 2001.

John Asker, Chaim Fershtman, Jihye Jeon, and Ariel Pakes. A computational framework for analyzing dynamic auctions: The market impact of information sharing. *The RAND Journal of Economics*, 51(3):805–839, 2020.

Lawrence M. Ausubel and Raymond J. Deneckere. Reputation in bargaining and durable goods monopoly. *Econometrica*, 57(3):511–531, 1989.

Leemon Baird. Residual algorithms: Reinforcement learning with function approximation. In *Proceedings of the Twelfth International Conference on Machine Learning*, pages 30–37. Morgan Kaufmann, 1995.

Richard Bellman. *Dynamic Programming*. Princeton University Press, 1957.

Dimitri P. Bertsekas. Lessons from AlphaZero for optimal, model predictive, and adaptive control. *Athena Scientific Reports*, 2021.

Dimitri P. Bertsekas. Newton's method for reinforcement learning and model predictive control. *Results in Control and Optimization*, 7:100121, 2022.

Gianluca Brero, Alon Eden, Matthias Gerstgrasser, David C. Parkes, and Duncan Rheingans-Yoo. Reinforcement learning of sequential price mechanisms. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 13662–13670, 2021.

George W. Brown. Iterative solution of games by fictitious play, 1951.

Noam Brown and Tuomas Sandholm. Superhuman ai for heads-up no-limit poker: Libratus beats top professionals. *Science*, 359(6374):418–424, 2018.

Noam Brown and Tuomas Sandholm. Superhuman ai for multiplayer poker. *Science*, 365 (6456):885–890, 2019.

Noam Brown, Adam Lerer, Sam Gross, and Tuomas Sandholm. Deep counterfactual regret minimization. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, volume 97, pages 793–802. PMLR, 2019.

Jeremy I. Bulow. Durable-goods monopolists. *Journal of Political Economy*, 90(2):314–332, 1982.

Shicong Cen, Chen Cheng, Yuxin Chen, Yuting Wei, and Yuejie Chi. Fast global convergence of natural policy gradient methods with entropy regularization. *Operations Research*, 70(4), 2022.

Nicolo Cesa-Bianchi, Claudio Gentile, and Yishay Mansour. Regret minimization for reserve prices in second-price auctions. *IEEE Transactions on Information Theory*, 61 (1):549–564, 2015.

Yiwei Chen, Guillermo Gallego, and Tao Li. A reinforcement learning approach for hotel revenue management. *Journal of Operations Management*, 2023.

Paul F. Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

Andrew J. Clark and Herbert Scarf. Optimal policies for a multi-echelon inventory problem. *Management Science*, 6(4):475–490, 1960.

Ronald H. Coase. Durability and monopoly. *Journal of Law and Economics*, 15(1): 143–149, 1972.

Bryan S. Covarrubias, Alexander Zentefis, and Jose M. Abiseid. Collusion with deep reinforcement learning. Technical Report w30283, National Bureau of Economic Research, July 2022.

Constantinos Daskalakis, Paul W. Goldberg, and Christos H. Papadimitriou. The complexity of computing a nash equilibrium. *SIAM Journal on Computing*, 39(1):195–259, 2009.

Chaim Fershtman and Ariel Pakes. Dynamic games with asymmetric information: A framework for empirical work. *The Quarterly Journal of Economics*, 127(4):1611–1661, 2012.

Guillermo Gallego and Garrett van Ryzin. Optimal dynamic pricing of inventories with stochastic demand over finite horizons. *Management Science*, 40(8):999–1020, 1994.

Jim Gao. Machine learning applications for data center optimization. Technical report, Google, 2014.

Joren Gijsbrechts, Robert N. Boute, Jan A. Van Mieghem, and Dennis J. Zhang. Can deep reinforcement learning improve inventory management? Performance on dual sourcing, lost sales, and multi-echelon problems. *Manufacturing & Service Operations Management*, 24(3):1349–1368, 2022.

John C. Gittins. Bandit processes and dynamic allocation indices. *Journal of the Royal Statistical Society: Series B (Methodological)*, 41(2):148–164, 1979.

Christian Gourieroux, Alain Monfort, and Eric Renault. Indirect inference. *Journal of Applied Econometrics*, 8(S1):S85–S118, 1993.

Vikas Goyal, Negin Jain, and Aleksandrs Slivkins. Online learning for multi-product dynamic pricing with multinomial logit choice model. *Operations Research*, 2022.

Faruk Gul, Hugo Sonnenschein, and Robert Wilson. Foundations of dynamic monopoly and the Coase conjecture. *Journal of Economic Theory*, 39(1):155–190, 1986.

Yanjun Guo, Yash Kanoria, and David Simchi-Levi. Multi-agent reinforcement learning in dynamic pricing games. *Management Science*, 2023.

Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. *Proceedings of the 35th International Conference on Machine Learning*, pages 1861–1870, 2018.

Xiao Han, Weijian Zhang, Jiaxin Wang, Fan Zhang, and Jieping Ye. A better match for drivers and riders: Reinforcement learning at Lyft. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 2927–2936, 2022.

Johannes Heinrich and David Silver. Deep reinforcement learning from self-play in imperfect-information games. *arXiv preprint arXiv:1603.01121*, 2016.

Brett Hollenbeck. Horizontal mergers and innovation in concentrated industries. *Quantitative Marketing and Economics*, 2019.

V. Joseph Hotz and Robert A. Miller. Conditional choice probabilities and the estimation of dynamic models. *The Review of Economic Studies*, 60(3):497–529, 1993.

Ronald A. Howard. *Dynamic Programming and Markov Processes*. The Technology Press of M.I.T. and John Wiley and Sons, New York, NY, 1960.

Yingyao Hu and Zhangyi Yang. Structural estimation with policy gradient methods. *Working Paper*, 2025.

Mitsuru Igami. Artificial intelligence as structural estimation: Deep blue, bonanza, and alphago. *The Econometrics Journal*, 23(3):S1–S24, 2020.

Tommi Jaakkola, Michael I. Jordan, and Satinder P. Singh. On the convergence of stochastic iterative dynamic programming algorithms. In *Advances in Neural Information Processing Systems 6*, pages 703–710. Morgan Kaufmann, 1994.

Sham M. Kakade. A natural policy gradient. *Advances in Neural Information Processing Systems*, 14, 2001.

Michael Kearns, Yishay Mansour, and Andrew Y. Ng. A sparse sampling algorithm for near-optimal planning in large Markov decision processes. *Machine Learning*, 49(2–3): 193–208, 2002.

David L. Kleinman. On an iterative technique for Riccati equation computations. *IEEE Transactions on Automatic Control*, 13(1):114–115, 1968.

Vijay R. Konda and John N. Tsitsiklis. Actor-critic algorithms. In *Advances in Neural Information Processing Systems 12*, pages 1008–1014. MIT Press, 2000.

Tomasz Korbak, Ethan Perez, and Christopher L. Buckley. RL with KL penalties is better viewed as Bayesian inference. *arXiv preprint arXiv:2205.11275*, 2022.

Nevena Lazic, Tyler Lu, Craig Boutilier, Moonkyung Ryu, Eehern Wong, Binz Roy, and Greg Imwalle. Data center cooling using model-predictive control. In *Advances in Neural Information Processing Systems*, volume 31, 2018.

Minne Li, Zhiwei Qin, Yan Jiao, Yaodong Yang, Zheng Gong, Jun Wang, Changjie Wang, Gauge Wu, and Jieping Ye. Efficient ridesharing order dispatching with mean field multi-agent reinforcement learning. In *Proceedings of the 2019 World Wide Web Conference (WWW)*, pages 983–994, 2019.

Nikolay Lomys and Luca Magnolfi. Estimation of games under no regret: Structural econometrics for ai. Working Paper NET Institute Working Paper No. 24-05, Social Science Research Network (SSRN), 2024. Available at SSRN: https://ssrn.com/abstract=4717195 or http://dx.doi.org/10.2139/ssrn.4717195.

Daniel McFadden. Conditional logit analysis of qualitative choice behavior. In Paul Zarembka, editor, *Frontiers in Econometrics*, pages 105–142. Academic Press, 1974. Working paper circulated 1973.

Kanishka Misra, Eric M. Schwartz, and Jacob Abernethy. Dynamic online pricing with incomplete information using multiarmed bandit experiments. *Marketing Science*, 38 (2):226–252, 2019.

Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015.

Volodymyr Mnih, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *Proceedings of the 33rd International Conference on Machine Learning*, pages 1928–1937. PMLR, 2016.

Jonas Mueller, Vasilis Syrgkanis, and Matt Taddy. Low-rank bandit methods for high-dimensional dynamic pricing. *Advances in Neural Information Processing Systems*, 32, 2019.

Roger B. Myerson. Optimal auction design. *Mathematics of Operations Research*, 6(1): 58–73, 1981.

Yuriy Nevmyvaka, Yi Feng, and Michael Kearns. Reinforcement learning for optimized trade execution. In *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, pages 673–680, 2006.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35, pages 27730–27744, 2022.

Ariel Pakes and Paul McGuire. Computing Markov-perfect Nash equilibria: Numerical implications of a dynamic differentiated product model. *RAND Journal of Economics*, 25(4):555–589, 1994.

Moshe A. Pollatschek and Benjamin Avi-Itzhak. Algorithms for stochastic games with geometrical interpretation. *Management Science*, 15(7):399–415, 1969.

Martin L. Puterman and Shelby L. Brumelle. On the convergence of policy iteration in stationary dynamic programming. *Mathematics of Operations Research*, 4(1):60–69, 1979.

Marek Pycia and Peter Troyan. A theory of simplicity in games and mechanism design. *Econometrica*, 91(4):1495–1526, 2023.

Zhiwei Qin, Xiaocheng Tang, Yan Jiao, Fan Zhang, Zhe Xu, Hongtu Zhu, and Jieping Ye. Ride-hailing order dispatching at DiDi via reinforcement learning. *INFORMS Journal on Applied Analytics*, 51(3):272–286, 2021.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

Sai Srivatsa Ravindranath, Zhe Feng, Di Wang, Manzil Zaheer, Aranyak Mehta, and David C. Parkes. Deep reinforcement learning for sequential combinatorial auctions. Submitted to ICLR 2025, 2024. Available at https://openreview.net/forum?id=SVd9Ffcdp8.

Robert A. Rescorla and Allan R. Wagner. A theory of Pavlovian conditioning: Variations in the effectiveness of reinforcement and nonreinforcement. In Abraham H. Black and William F. Prokasy, editors, *Classical Conditioning II: Current Research and Theory*, pages 64–99. Appleton-Century-Crofts, 1972.

Herbert Robbins. Some aspects of the sequential design of experiments. *Bulletin of the American Mathematical Society*, 58(5):527–535, 1952.

John Rust. Optimal replacement of gmc bus engines: An empirical model of harold zurcher. *Econometrica*, 55(5):999–1033, 1987.

John Rust and Pranjal Rawat. Structural econometrics and inverse reinforcement learning: Inferring preferences and beliefs from human behavior. Working Paper, Georgetown University, 2026a.

John Rust and Pranjal Rawat. Structural econometrics and inverse reinforcement learning: Inferring preferences and beliefs from human behavior. Working Paper, 2026b.

Arthur L. Samuel. Some studies in machine learning using the game of checkers. *IBM Journal of Research and Development*, 3(3):210–229, 1959.

John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. Trust region policy optimization. *Proceedings of the 32nd International Conference on Machine Learning*, pages 1889–1897, 2015.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Claude E. Shannon. Programming a computer for playing chess. *Philosophical Magazine*, 41(314):256–275, 1950.

David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, Sander Dieleman, Dominik Grewe, John Nham, Nal Kalchbrenner, Ilya Sutskever, Timothy Lillicrap, Madeleine Leach, Koray Kavukcuoglu, Thore Graepel, and Demis Hassabis. Mastering the game of go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016.

David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, Yutian Chen, Timothy Lillicrap, Fan Hui, Laurent Sifre, George van den Driessche, Thore Graepel, and Demis Hassabis. Mastering the game of go without human knowledge. *Nature*, 550(7676):354–359, 2017.

Satinder Singh, Tommi Jaakkola, Michael L Littman, and Csaba Szepesvári. Convergence results for single-step on-policy reinforcement-learning algorithms. *Machine learning*, 38:287–308, 2000.

Satinder P. Singh and Richard C. Yee. An upper bound on the loss from approximate optimal-value functions. *Machine Learning*, 16(3):227–233, 1994.

Nisan Stiennon, Long Ouyang, Jeffrey Wu, Daniel Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul F. Christiano. Learning to summarize with human feedback. In *Advances in Neural Information Processing Systems*, volume 33, pages 3008–3021, 2020.

Nancy L. Stokey. Rational expectations and durable goods pricing. *Bell Journal of Economics*, 12(1):112–128, 1981.

Nancy L. Stokey, Robert E. Lucas, and Edward C. Prescott. *Recursive Methods in Economic Dynamics*. Harvard University Press, Cambridge, MA, 1989.

Richard S. Sutton. Learning to predict by the methods of temporal differences. *Machine Learning*, 3(1):9–44, 1988. doi: 10.1023/A:1022633531479.

Richard S. Sutton. Integrated architectures for learning, planning, and reacting based on approximating dynamic programming. In *Proceedings of the Seventh International Conference on Machine Learning*, pages 216–224. Morgan Kaufmann, 1990.

Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. The MIT Press, 2nd edition, 2018.

Richard S. Sutton, David McAllester, Satinder Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. In *Advances in Neural Information Processing Systems*, volume 12. MIT Press, 2000.

Richard S. Sutton, Hamid Reza Maei, Doina Precup, Shalabh Bhatnagar, David Silver, Csaba Szepesvári, and Eric Wiewiora. Fast gradient-descent methods for temporal-difference learning with linear function approximation. In *Proceedings of the 26th International Conference on Machine Learning*, pages 993–1000. ACM, 2009.

Oskari Tammelin. Solving large imperfect information games using cfr+. *arXiv preprint arXiv:1407.5042*, 2014.

Gerald Tesauro. Td-gammon, a self-teaching backgammon program, achieves master-level play. *Neural Computation*, 6(2):215–219, 1994.

Edward L. Thorndike. *Animal Intelligence: An Experimental Study of the Associative Processes in Animals*. Macmillan, 1898. Psychological Review Monograph Supplements, No. 8.

John N. Tsitsiklis and Benjamin Van Roy. Analysis of temporal-difference learning with function approximation. In *Advances in Neural Information Processing Systems 9 (NIPS)*, 1997.

Yongfeng Wang, Kangyi Ouyang, Chunyan Huang, Jian Chen, Yanli Liu, and Ying Wei. Aliexpress dynamic pricing: A learning approach. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 2547–2555, 2019a.

Zhaodong Wang, Zhiwei Qin, Xiaocheng Tang, Jieping Ye, and Hongtu Zhu. Reinforcement learning for order dispatching: A ride-sharing perspective. *arXiv preprint arXiv:1905.11566*, 2019b.

Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8(3–4): 279–292, 1992.

Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8:229–256, 1992.

Di Wu, Xiujun Chen, Xun Yang, Hao Wang, Qing Tan, Xiaoxun Zhang, Jian Xu, and Kun Gai. Budget constrained bidding by model-free reinforcement learning in display advertising. In *Proceedings of the 27th ACM International Conference on Information and Knowledge Management (CIKM)*, pages 1443–1451, 2018.

Yining Xu, Zhe Wang, and Yuchen Yu. Logarithmic regret in feature-based dynamic pricing. *Advances in Neural Information Processing Systems*, 34, 2021.

Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Sainbayar Sukhbaatar, Jing Xu, and Jason Weston. Self-rewarding language models. *arXiv preprint arXiv:2401.10020*, 2024.

Daniel M. Ziegler, Nisan Stiennon, Jeffrey Wu, Tom B. Brown, Alec Radford, Dario Amodei, Paul Christiano, and Geoffrey Irving. Fine-tuning language models from human preferences. *arXiv preprint arXiv:1909.08593*, 2019.

Martin Zinkevich, Michael Johanson, Michael Bowling, and Carmelo Piccione. Regret minimization in games with incomplete information. In *Advances in Neural Information Processing Systems*, volume 20, 2008.